

MERN Stack Knowledge Base

Authored guides and connected practical study

This offline edition contains the full authored text of the 25 canonical guides, including the 375 concept definitions, priorities, mental models, examples, exercises, answered interview prompts, and resource links. Depth labels distinguish worked examples from reference definitions. It does not reproduce external publisher books or certify every referenced API.

Use the repository for runnable source, browser practice, project requirements, current corrections, and downloads. Code lines may wrap visually here; copy runnable code from its source file.

Exported 2026-10-09. Source review dates remain the dates recorded in the catalog.

Contents

TypeScript and validated application boundaries

Git history, collaboration, and code review

React identity, state, effects, and resilient interfaces

Local state, shared state, and server-state ownership

Next.js routing, rendering, and server boundaries

Internet, HTTP, and the browser

Semantic HTML, forms, and accessible documents

CSS layout, cascade, and responsive design

JavaScript values, scope, functions, and collections

Promises, event loops, and bounded concurrency

DOM, events, and browser APIs

Node.js runtime, resources, and asynchronous services

Express services, validation, and protected REST routes

MongoDB modeling, querying, and persisted invariants

Mongoose schemas, models, and database behavior

Relational modeling, SQL queries, and transactions

Redis caching, messaging, and distributed coordination

Cloud deployment, scaling, and recovery choices

Maintainable code, architecture, and observability

System design, consistency, and failure tradeoffs

REST contracts, GraphQL, and API evolution

Authentication, authorization, and web security boundaries

WebSockets, SSE, and recoverable real-time delivery

Behavioral testing across units, databases, and browsers

Containers, CI/CD, and operating a web service

TypeScript and validated application boundaries

Priority: P1 — ☐ Highly Important

Difficulty: Intermediate

Importance: 4/5

Prerequisites: JavaScript values, scope, functions, and collections

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

TypeScript checks relationships between values before execution; the emitted JavaScript still needs runtime validation.

Concept reference and priorities

Fundamentals

P1 · ☐ Highly Important · reference

TypeScript checks relationships between values before execution; the emitted JavaScript still needs runtime validation.

Types

P1 · ☐ Highly Important · worked-example

Types describe permitted values and operations. Prefer precise domain types to pervasive any.

Interfaces

P1 · ☐ Highly Important · reference

Interfaces describe structural shapes and can support declaration merging.

Type aliases

P1 · ☐ Highly Important · reference

Aliases name type expressions such as unions, tuples, and mapped types.

Unions

P1 · ☐ Highly Important · worked-example

A union permits one of several alternatives; discriminate valid application states with a stable field.

Intersections

P2 · Useful · reference

An intersection requires the combined constraints of multiple types; incompatible members can produce impossible types.

Generics

P1 · ☐ Highly Important · reference

Generics preserve a relationship between inputs and outputs rather than discarding information through any.

Utility types

P2 · Useful · reference

Utilities such as Pick, Omit, Partial, and Record transform type structure; they do not validate data.

Narrowing

P1 · ☐ Highly Important · worked-example

Control-flow analysis refines a type after checks such as typeof or a discriminant test.

Type guards

P1 · ☐ Highly Important · reference

A guard claims a runtime check establishes a type; an incorrect guard can create false confidence.

Enums

P2 · Useful · reference

Enums can emit runtime objects; literal unions are often enough when only a compile-time set is required.

Modules

P1 · ☐ Highly Important · reference

Explicit imports and exports coordinate types and values; type-only imports avoid unnecessary runtime dependencies.

Classes

P2 · Useful · reference

Access modifiers and implements help model intent; static type constraints are not a substitute for runtime authority.

Advanced types

P3 · Advanced / Specialized · reference

Conditional, mapped, indexed-access, and template-literal types express complex relationships at a readability cost.

TypeScript with React

P1 · ☐ Highly Important · worked-example

Type props, events, and state according to their actual roles; unions can prevent conflicting request states.

TypeScript with Node

P1 · ☐ Highly Important · reference

Match compiler module settings to Node's runtime package and import rules.

TypeScript with Express

P1 · ☐ Highly Important · reference

Type checked service inputs after validating the request; request generics alone do not authenticate a payload.

Full-stack TypeScript

P1 · ☐ Highly Important · reference

Share domain contracts while preserving server-owned fields and independent runtime boundary validation.

Why Does It Exist?

A loading boolean, optional data, and optional error permit combinations that the application cannot meaningfully display. Explicit types make these states visible before the browser runs.

How Does It Work?

Treat unknown network data as unknown until checked. Narrow a discriminated union before accessing variant fields. Use strict checking and prefer a small understandable model over a clever generic that hides the contract. Generated API types can reduce drift, but they describe expected data rather than prove what arrived.

Examples

1. Trace the contract

```
type Load<T> =  
  | {status:'loading'}  
  | {status:'success'; data:T}  
  | {status:'error'; message:string};  
function size(state: Load<string[]>): number {  
  if (state.status === 'success') return state.data.length;  
  return 0;  
}  
console.log(size({status:'success', data:['mern']}));
```

Expected behavior and runtime: Compiled with a TypeScript compiler, the log is 1. Accessing data before narrowing is a type error. This snippet is a type-design example, not a claim that the JavaScript workspace has already been migrated.

2. Extend and stress the contract

Intermediate: narrow the example and add an empty result state. Advanced: implement an exhaustiveness check when a variant changes. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

The type checker tracks structural compatibility and control flow. Most type annotations disappear in emitted JavaScript; an assertion does not insert a runtime check. Type complexity can also increase editor and build cost.

Real-World Usage

Migrate one task response and one React request state first. Keep validation at the HTTP edge and return typed domain results to services.

Common Mistakes

- Casting `response.json()` to a trusted domain object.
- Using `Partial<User>` as a writable API payload, including privileged fields.
- Sharing server secrets or server-only modules through a client import.

Best Practices

Migrate one task response and one React request state first. Keep validation at the HTTP edge and return typed domain results to services. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: distinguish a literal union from a string.
- Intermediate: narrow the example and add an empty result state.
- Advanced: implement an exhaustiveness check when a variant changes.
- Challenge: compile and run a small boundary parser with invalid JSON inputs.

Mini Project

Create a strict TypeScript slice for bookmark validation and its UI state; acceptance includes compile checks and runtime rejection of malformed input.

Project ladder and implementation status

Interview Questions

Beginner: Does TypeScript validate an API response?

No. Validate the received runtime value at the boundary.

Intermediate: Why prefer unknown over any at an input edge?

Unknown requires a check before use, while any disables useful constraints.

Advanced: Can an assertion prove authorization?

No. Authorization depends on trusted runtime identity and resource scope.

Scenario: A field is missing despite successful compilation.

Reproduce the received payload and add a runtime validator; inspect contract drift.

Debugging: Two request-state booleans disagree.

Replace invalid combinations with a discriminated union and explicit transitions.

Output-based: What does the worked example produce, and under which runtime?

Compiled with a TypeScript compiler, the log is 1. Accessing data before narrowing is a type error. This snippet is a type-design example, not a claim that the JavaScript workspace has already been migrated.

Coding: What would you implement and verify next?

Create a strict TypeScript slice for bookmark validation and its UI state; acceptance includes compile checks and runtime rejection of malformed input.

Architecture and production: Which internal boundary needs evidence?

The type checker tracks structural compatibility and control flow. Most type annotations disappear in emitted JavaScript; an assertion does not insert a runtime check. Type complexity can also increase editor and build cost.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

React identity, state, effects, and resilient interfaces, Express services, validation, and protected REST routes, Maintainable code, architecture, and observability

Next: React identity, state, effects, and resilient interfaces

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Git history, collaboration, and code review

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: No programming prerequisites; basic file and browser use.

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Commits identify snapshots and parent relationships; the working tree and staging area represent different states.

Concept reference and priorities

Git fundamentals

P0 · Essential / Master · worked-example

Commits identify snapshots and parent relationships; the working tree and staging area represent different states.

Branching

P0 · Essential / Master · worked-example

A branch is a movable name pointing to a commit, not a separate copy of every file.

Merging

P0 · Essential / Master · reference

A merge combines histories, potentially creating a commit with multiple parents.

Rebasing

P2 · Useful · reference

Rebase replays commits onto a new base and changes their identities; coordinate before rewriting shared history.

Stashing

P2 · Useful · reference

Stash stores selected work temporarily; understand whether untracked files are included and verify restoration.

Cherry-pick

P2 · Useful · reference

Cherry-pick applies a commit's change on another history and normally creates a new commit identity.

Reset

P2 · Useful · reference

Reset moves a reference and can alter the index or working tree depending on mode; inspect before using destructive modes.

Revert

P1 · ☐ Highly Important · reference

Revert creates a new commit that reverses an earlier change while retaining shared history.

Git workflows

P1 · ☐ Highly Important · reference

A workflow defines branches, review, checks, and release decisions; choose it for the team rather than fashion.

Pull requests

P0 · Essential / Master · reference

A pull request proposes a branch difference for review and collaboration.

Code review

P0 · Essential / Master · worked-example

Review checks behavior, maintainability, evidence, security boundaries, and scope rather than only formatting.

GitHub Actions

P1 · ☐ Highly Important · reference

Workflows run configured jobs on events; least-privilege permissions and untrusted-input handling matter.

Open-source contribution

P1 · ☐ Highly Important · reference

Read contribution rules, preserve provenance, describe the change, and supply reproducible verification.

Why Does It Exist?

Version history lets a team explain, review, and recover a change. A good diff and honest checks are as useful to maintainers as a polished final screen.

How Does It Work?

Inspect status first, stage intended changes, inspect the staged diff, and commit one coherent behavior. Fetch updates before comparing with the remote. A conflict asks you to reconcile intent: choosing every incoming line mechanically can discard necessary behavior. Prefer a revert for a published mistake when the team needs shared history preserved.

Examples

1. Trace the contract

```
git status --short
git switch -c feature/request-validation
git diff
git add server/validation.js
git diff --cached
git commit -m "Validate task titles at the API boundary"
```

Expected behavior and runtime: Run in a disposable repository with that file; the staged diff includes the selected file, and the commit records that snapshot. These commands do not push or merge anything.

2. Extend and stress the contract

Intermediate: merge two edits to a small function and verify intent. Advanced: compare a merge and a rebase in a disposable repository. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Git objects are content-addressed and commits connect snapshots through parents. A branch name and a remote-tracking name can diverge. Line-ending normalization is controlled by attributes; binary artifacts must not be transformed as text.

Real-World Usage

Contribute one debugging challenge with a focused PR, expected behavior, reproduction, regression evidence, and source verification.

Common Mistakes

- Resetting a shared branch to hide an error without coordination.
- Committing .env or private progress data.
- Assuming a clean working tree means the code was tested.

Best Practices

Contribute one debugging challenge with a focused PR, expected behavior, reproduction, regression evidence, and source verification. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: explain working, staged, and committed versions of one file.
- Intermediate: merge two edits to a small function and verify intent.
- Advanced: compare a merge and a rebase in a disposable repository.
- Challenge: revert a behavioral change and keep unrelated later work.

Mini Project

Create a contribution rehearsal repository with two branches, one conflict, one regression check, and a reviewed change description.

Project ladder and implementation status

Interview Questions

Beginner: What is a branch?

A movable reference to a commit.

Intermediate: How does revert differ from reset?

Revert adds a compensating commit; reset moves a reference and may alter local states.

Advanced: Why are rebased commit hashes different?

Their parents and often metadata change, yielding new commit objects.

Scenario: A secret was committed.

Revoke or rotate it first, then coordinate history and log cleanup through the security process.

Debugging: A file is absent from a commit.

Inspect status and the staged diff; an unstaged change is not part of the new commit.

Output-based: What does the worked example produce, and under which runtime?

Run in a disposable repository with that file; the staged diff includes the selected file, and the commit records that snapshot. These commands do not push or merge anything.

Coding: What would you implement and verify next?

Create a contribution rehearsal repository with two branches, one conflict, one regression check, and a reviewed change description.

Architecture and production: Which internal boundary needs evidence?

Git objects are content-addressed and commits connect snapshots through parents. A branch name and a remote-tracking name can diverge. Line-ending normalization is controlled by attributes; binary artifacts must not be transformed as text.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Maintainable code, architecture, and observability, Containers, CI/CD, and operating a web service

Next: Maintainable code, architecture, and observability

Preserved lessons: notes/git.md

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

React identity, state, effects, and resilient interfaces

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: Semantic HTML, forms, and accessible documents, JavaScript values, scope, functions, and collections, Promises, event loops, and bounded concurrency, DOM, events, and browser APIs

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

React describes interfaces as components whose output depends on inputs and state.

Concept reference and priorities

Fundamentals

P0 · Essential / Master · reference

React describes interfaces as components whose output depends on inputs and state.

JSX

P0 · Essential / Master · reference

JSX describes element structure and is transformed before execution; it is not an HTML string.

Components

P0 · Essential / Master · reference

Components encapsulate presentation and behavior; keep rendering pure so repeated evaluation remains safe.

Props

P0 · Essential / Master · reference

Props are inputs from a parent and should not be mutated by the receiving component.

State

P0 · Essential / Master · worked-example

State records facts the component owns. Derive values that can be computed from existing facts.

Events

P0 · Essential / Master · worked-example

Event handlers express user intent and can trigger state changes or commands.

Forms

P0 · Essential / Master · reference

Form state, validation, submission, and recovery need distinct behavior; preserve drafts when requests fail.

Conditional rendering

P0 · Essential / Master · reference

Render a clear representation for each valid state, including loading, empty, error, and success.

Lists

P0 · Essential / Master · reference

Transform data into elements without changing item identity as order changes.

Keys

P0 · Essential / Master · reference

Stable keys let React identify siblings across updates; random or index-based keys can lose the correct local state.

Hooks

P0 · Essential / Master · worked-example

Hooks connect React capabilities to function components under ordering rules.

useState

P0 · Essential / Master · worked-example

useState provides a state snapshot and update function; functional updates compose against previous state.

useEffect

P0 · Essential / Master · reference

Effects synchronize with external systems and need correct dependencies and cleanup.

useRef

P1 · ☐ Highly Important · reference

A ref persists a mutable value without scheduling rendering; do not use it to conceal missing UI state.

useMemo

P2 · Useful · reference

Memoization can avoid recomputation when dependencies are equal; measure benefit and avoid relying on cache for correctness.

useCallback

P2 · Useful · reference

A cached function identity can help specific memoized boundaries; it is not a general speed switch.

useContext

P1 · ☐ Highly Important · reference

Context passes a value through a subtree; changing that value can affect consuming components.

Custom hooks

P1 · ☐ Highly Important · reference

A custom hook reuses stateful logic while each caller has its own hook state unless an external store is used.

Component architecture

P1 · [□](#) **Highly Important** · [reference](#)

Separate reusable presentation, domain state, and external communication where this simplifies change.

Routing

P1 · [□](#) **Highly Important** · [reference](#)

Routing maps location to views and defines navigation, parameters, and loading boundaries.

React Router

P1 · [□](#) **Highly Important** · [reference](#)

React Router offers modes with different routing and data behavior; choose a mode deliberately and consult its matching docs.

Data fetching

P0 · **Essential / Master** · [reference](#)

Associate responses with request identity and model caching, invalidation, cancellation, and errors.

Error handling

P0 · **Essential / Master** · [reference](#)

Distinguish render errors, rejected asynchronous work, validation failures, and conflict recovery.

Performance

P1 · [□](#) **Highly Important** · [reference](#)

Profile actual interaction; list size, expensive work, and network waterfalls often matter more than blanket memoization.

Lazy loading

P2 · **Useful** · [reference](#)

Load code for a boundary when needed, then provide loading and failure behavior for that boundary.

Suspense

P2 · **Useful** · [reference](#)

Suspense coordinates supported pending resources; arbitrary effect-based fetching does not automatically integrate with it.

Server/client concepts

P2 · **Useful** · [reference](#)

Server and client components have different capabilities and bundle boundaries supplied by a compatible framework.

Testing

P0 · **Essential / Master** · [reference](#)

Test visible behavior and accessible interaction, then run important workflows in a real browser.

Accessibility

P0 · **Essential / Master** · [reference](#)

Use native roles, labels, focus management, and keyboard behavior through state transitions.

Production architecture

P1 · ☐ Highly Important · reference

Define data ownership, request contracts, error recovery, bundle boundaries, and operating evidence before multiplying abstractions.

Why Does It Exist?

A changing interface needs a consistent relationship between facts, displayed values, and commands. React makes this relationship explicit when components remain pure and state has a clear owner.

How Does It Work?

A render sees a snapshot. Updating state schedules later rendering; it does not change an old closure in place. Derive totals during render and put a submit command in the submit handler. An effect connects to something outside rendering; its cleanup undoes that connection. Keep item drafts associated with stable ids rather than array positions.

Examples

1. Trace the contract

```
function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => {
    setCount(c => c + 1);
    setCount(c => c + 1);
  }}>Count {count}</button>;
}
```

Expected behavior and runtime: In a React application importing `useState`, each click adds two. Replacing both updates with `setCount(count + 1)` computes the same next value from the current snapshot and adds one.

2. Extend and stress the contract

Intermediate: move a derived value out of state and preserve a draft after a failed request. Advanced: demonstrate effect cleanup under remounting and changed dependencies. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

React evaluates components, reconciles identity, and commits host changes. Rendering can happen more than once; side effects during rendering can therefore duplicate external work. Development Strict Mode can expose missing cleanup by exercising additional setup/cleanup behavior.

Real-World Usage

Inspect the MERN workspace form drafts and conflict refresh. Extend it with search that rejects stale results and a loading representation that does not erase unsent input.

Common Mistakes

- Using an effect to copy a computable total into another state variable.
- Initiating payment or creation merely because a component rendered.
- Using the sorted row index as an editable row key.

Best Practices

Inspect the MERN workspace form drafts and conflict refresh. Extend it with search that rejects stale results and a loading representation that does not erase unsent input. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: explain the counter output using snapshots.
- Intermediate: move a derived value out of state and preserve a draft after a failed request.
- Advanced: demonstrate effect cleanup under remounting and changed dependencies.
- Challenge: reproduce an async race and prove the latest-query invariant in a browser test.

Mini Project

Build the timed search and table exercises in the interview handbook. Acceptance includes out-of-order responses, empty and failure states, stable row identity, and keyboard operation.

Project ladder and implementation status

Interview Questions

Beginner: How do props differ from state?

Props come from a parent; state records facts owned by the relevant component or model.

Intermediate: When is an effect appropriate?

When synchronizing with an external system, with cleanup and dependencies that describe the connection.

Advanced: Does memoization establish correctness?

No. The application must remain correct when computation is repeated or a cache is unavailable.

Scenario: Sorting moves the wrong draft.

Use stable domain keys and decide whether drafts are row-local or held in a keyed parent model.

Debugging: A request loops forever.

Inspect unstable dependencies and state changes in the effect; simplify the state model before suppressing warnings.

Output-based: What does the worked example produce, and under which runtime?

In a React application importing `useState`, each click adds two. Replacing both updates with `setCount(count + 1)` computes the same next value from the current snapshot and adds one.

Coding: What would you implement and verify next?

Build the timed search and table exercises in the interview handbook. Acceptance includes out-of-order responses, empty and failure states, stable row identity, and keyboard operation.

Architecture and production: Which internal boundary needs evidence?

React evaluates components, reconciles identity, and commits host changes. Rendering can happen more than once; side effects during rendering can therefore duplicate external work. Development Strict Mode can expose missing cleanup by exercising additional setup/cleanup behavior.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Local state, shared state, and server-state ownership, TypeScript and validated application boundaries, Next.js routing, rendering, and server boundaries, Behavioral testing across units, databases, and browsers

Next: Local state, shared state, and server-state ownership

Preserved lessons: notes/react.md, 05-react/notes.md

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Local state, shared state, and server-state ownership

Priority: P1 — ☐ Highly Important

Difficulty: Intermediate

Importance: 4/5

Prerequisites: React identity, state, effects, and resilient interfaces

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Keep a fact near its users until a real sharing requirement appears.

Concept reference and priorities

Local state

P0 · Essential / Master · worked-example

Keep a fact near its users until a real sharing requirement appears.

Context

P1 · ☐ Highly Important · reference

Context transports a shared value through a subtree; it does not automatically manage remote caching or normalize entities.

Redux

P2 · Useful · reference

Redux centralizes predictable transitions through actions and reducers; it has an ecosystem for complex shared application state.

Redux Toolkit

P1 · ☐ Highly Important · reference

Redux Toolkit provides the recommended Redux setup and utilities, reducing hand-written store boilerplate.

Zustand

P2 · Useful · reference

Zustand provides an external store with selectors; subscriptions and ownership still need intentional design.

Server state

P0 · Essential / Master · worked-example

Server state has remote authority, freshness, invalidation, and concurrent-writer concerns beyond local component state.

TanStack Query

P1 · ☐ Highly Important · reference

TanStack Query coordinates asynchronous server-state caching and lifecycle; query keys and invalidation define correctness.

When to use which approach

P0 · Essential / Master · worked-example

Choose local state, context, an external store, or a query cache according to ownership and coordination needs.

Why Does It Exist?

Copying every API response into several stores creates drift. A draft and a saved server record have different owners and different conflict rules.

How Does It Work?

Classify each fact before selecting a library: a temporary form draft is local; theme is shared UI configuration; a server list is cached remote state. A query key must include relevant parameters and scope. Invalidate or update cache after a mutation according to the authoritative response. Separate users and clear sensitive cache on session changes.

Examples

1. Trace the contract

```
const queryKey = ['bookmarks', userId, {search, page}];  
// Example contract for a query library: every changing read parameter is keyed.  
const draft = {title: '', url: ''};  
// Draft belongs to the form. Saved bookmarks belong to the server.
```

Expected behavior and runtime: These values illustrate ownership; no query library is invoked. Changing `userId`, `search`, or `page` must select a distinct read identity, and a failed write must not silently discard draft.

2. Extend and stress the contract

Intermediate: include all filter parameters in query identity. Advanced: design rollback or refetch after a 409 conflict. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Reducers describe transitions; external-store selectors determine subscriptions; query caches schedule freshness and refetching. A shared cache can deduplicate reads but cannot replace a server uniqueness index or version predicate.

Real-World Usage

Introduce a query cache into the workspace only after writing the read and invalidation contract. Compare network traces and retained form behavior before and after.

Common Mistakes

- Choosing Redux for every small component by default.
- Omitting the account id from a shared query key.
- Letting an optimistic cache update overwrite a later authoritative result.

Best Practices

Introduce a query cache into the workspace only after writing the read and invalidation contract. Compare network traces and retained form behavior before and after. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: classify ten UI facts by owner.
- Intermediate: include all filter parameters in query identity.
- Advanced: design rollback or refetch after a 409 conflict.
- Challenge: switch between two accounts and prove old private data is not reused.

Mini Project

Implement a server-state adapter for reading-list CRUD. Acceptance includes session changes, invalidation after deletion, failed mutation recovery, and one documented tradeoff between libraries.

Project ladder and implementation status

Interview Questions

Beginner: Is a query cache the database?

No. It is a client-side representation with freshness and invalidation behavior.

Intermediate: What belongs in a query key?

Every input that changes the represented read, including relevant account scope.

Advanced: Can a client store prevent two-device writes?

No. Persisted server concurrency rules must establish that invariant.

Scenario: One user sees cached data from another.

Separate account-scoped keys and clear or partition sensitive caches during session changes.

Debugging: A saved item never appears.

Check query identity, mutation result handling, invalidation, and server response before forcing arbitrary rerenders.

Output-based: What does the worked example produce, and under which runtime?

These values illustrate ownership; no query library is invoked. Changing userId, search, or page must select a distinct read identity, and a failed write must not silently discard draft.

Coding: What would you implement and verify next?

Implement a server-state adapter for reading-list CRUD. Acceptance includes session changes, invalidation after deletion, failed mutation recovery, and one documented tradeoff between libraries.

Architecture and production: Which internal boundary needs evidence?

Reducers describe transitions; external-store selectors determine subscriptions; query caches schedule freshness and refetching. A shared cache can deduplicate reads but cannot replace a server uniqueness index or version predicate.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

REST contracts, GraphQL, and API evolution, Maintainable code, architecture, and observability

Next: REST contracts, GraphQL, and API evolution

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Next.js routing, rendering, and server boundaries

Priority: P2 — Useful

Difficulty: Intermediate

Importance: 3/5

Prerequisites: React identity, state, effects, and resilient interfaces, REST contracts, GraphQL, and API evolution, Authentication, authorization, and web security boundaries

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Next.js supplies routing, rendering, and server integration around React; match documentation to the installed version and router.

Concept reference and priorities

Fundamentals

P2 · Useful · reference

Next.js supplies routing, rendering, and server integration around React; match documentation to the installed version and router.

Routing

P2 · Useful · reference

File conventions map URLs to pages, layouts, loading states, and handlers; parameters still need validation.

Layouts

P2 · Useful · reference

Layouts wrap route segments and can persist across navigation; do not assume every navigation remounts them.

Server Components

P2 · Useful · reference

Server Components can read server resources without sending their implementation to the client bundle.

Client Components

P2 · Useful · reference

Client boundaries enable interactive hooks and browser capabilities; their imported client dependency graph affects the bundle.

Data fetching

P2 · Useful · reference

Choose server or client fetching according to authority, latency, and interaction; define freshness explicitly.

Server Actions

P2 · Useful · reference

Server Actions execute server functions through framework transport and still require input validation and authorization.

API routes

P2 · Useful · worked-example

App Router Route Handlers expose HTTP methods; Pages Router API routes use a different convention.

Middleware

P3 · Advanced / Specialized · reference

In Next.js 16, the middleware file convention is deprecated in favor of proxy; it is distinct from Express middleware.

Authentication

P1 · ☐ Highly Important · reference

Resolve trusted identity and enforce resource authorization at the data or operation boundary, including actions and handlers.

Caching

P2 · Useful · reference

Caching behavior is version-sensitive; use explicit cache and revalidation choices rather than historical default assumptions.

Rendering strategies

P2 · Useful · reference

Choose when and where content is rendered based on personalized data, freshness, and distribution needs.

Static rendering

P2 · Useful · reference

Precomputed content can be distributed cheaply when its inputs and invalidation rules permit it.

Dynamic rendering

P2 · Useful · reference

Request-dependent content must preserve per-request identity and avoid unsafe shared caching.

Streaming

P3 · Advanced / Specialized · reference

Streaming sends ready portions before the whole response finishes; boundaries affect loading and error behavior.

Deployment

P2 · Useful · reference

Hosting must support the selected server features; a static export cannot provide arbitrary dynamic server behavior.

Performance

P2 · Useful · reference

Measure bundle size, waterfalls, caching, and useful rendered content before changing rendering strategy.

Why Does It Exist?

Server rendering can reduce client work and keep some dependencies on the server. It also introduces new data, cache, and authorization boundaries that a client-only application did not have.

How Does It Work?

Use the App Router terminology consistently. Keep read-only server access behind server modules, pass only intended serializable values across client boundaries, and validate route parameters. A route visibility check is not sufficient for authorization: direct calls to the protected operation still need scope checks. A shared cache must never mix personalized responses from different users.

Examples

1. Trace the contract

```
// app/api/ping/route.ts -- Next.js App Router file convention
export async function GET() {
  return Response.json({status:'ok'}, {
    headers:{'Cache-Control':'no-store'}
  });
}
```

Expected behavior and runtime: In an App Router project, GET /api/ping returns a JSON object. The explicit response header avoids browser/intermediary reuse; separately examine framework data/render caching for real reads.

2. Extend and stress the contract

Intermediate: mark one client boundary and inspect its dependency graph. Advanced: design cache keys and revalidation for public versus account-specific reads. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

The framework coordinates server rendering, React payloads, client hydration, and route navigation. A use client directive creates a module boundary, not a claim that every part is rendered exclusively in a browser. Next.js 16 names the pre-route convention proxy; inspect the versioned migration guide before adapting older tutorials.

Real-World Usage

Build a public reading page with a small interactive bookmark control. Explain which reads can be public and which writes require a session and resource ownership.

Common Mistakes

- Assuming a Server Action is inherently authorized.
- Putting secret-bearing values into props delivered to the client.
- Following an older cache-default or middleware tutorial without checking the installed version.

Best Practices

Build a public reading page with a small interactive bookmark control. Explain which reads can be public and which writes require a session and resource ownership. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: compare page, layout, and Route Handler responsibilities.
- Intermediate: mark one client boundary and inspect its dependency graph.

- Advanced: design cache keys and revalidation for public versus account-specific reads.
- Challenge: test a direct unauthorized server operation and an authenticated operation for another owner.

Mini Project

Create an optional Next.js adapter for the reading-list domain. Preserve the existing API contract; verify authorization, loading, hydration, cache separation, and deployment requirements.

Project ladder and implementation status

Interview Questions

Beginner: Is Next.js required for MERN?

No. It is an optional React framework with additional rendering and server capabilities.

Intermediate: Does hiding a page protect an action?

No. The operation itself must validate identity, input, and permissions.

Advanced: What is unsafe personalized caching?

A reusable response may expose one user's data to another if identity and private policy are omitted.

Scenario: An old tutorial uses `middleware.ts`.

Check the installed version; Next.js 16 deprecates that convention in favor of proxy.

Debugging: A static deployment loses an API feature.

Inspect whether the feature requires a server runtime instead of static export.

Output-based: What does the worked example produce, and under which runtime?

In an App Router project, `GET /api/ping` returns a JSON object. The explicit response header avoids browser/intermediary reuse; separately examine framework `data/render` caching for real reads.

Coding: What would you implement and verify next?

Create an optional Next.js adapter for the reading-list domain. Preserve the existing API contract; verify authorization, loading, hydration, cache separation, and deployment requirements.

Architecture and production: Which internal boundary needs evidence?

The framework coordinates server rendering, React payloads, client hydration, and route navigation. A `use client` directive creates a module boundary, not a claim that every part is rendered exclusively in a browser. Next.js 16 names the pre-route convention `proxy`; inspect the versioned migration guide before adapting older tutorials.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Local state, shared state, and server-state ownership, Cloud deployment, scaling, and recovery choices, Containers, CI/CD, and operating a web service

Next: Local state, shared state, and server-state ownership

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Internet, HTTP, and the browser

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: No programming prerequisites; basic file and browser use.

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

The Internet connects networks. The Web uses that infrastructure to exchange resources identified by URLs.

Concept reference and priorities

Internet fundamentals

P0 · Essential / Master · reference

The Internet connects networks. The Web uses that infrastructure to exchange resources identified by URLs.

How the Web works

P0 · Essential / Master · reference

A browser requests resources, interprets HTML and CSS, runs scripts, and sends later application requests.

Browser architecture

P2 · Useful · reference

Browser processes separate responsibilities such as rendering, networking, and isolation; their implementation varies by browser.

HTTP/HTTPS

P0 · Essential / Master · worked-example

HTTP defines request and response semantics. HTTPS carries HTTP over an authenticated, encrypted TLS connection.

DNS

P1 · ☐ Highly Important · reference

DNS resolves names into records used to locate services. DNS resolution and HTTP caching have different lifetimes.

TCP/IP basics

P2 · Useful · reference

IP routes packets; TCP provides a reliable ordered byte stream. HTTP/3 uses QUIC over UDP instead of TCP.

APIs

P0 · Essential / Master · worked-example

An API is a contract between software components. A web API exposes a network-accessible contract.

JSON

P0 · Essential / Master · worked-example

JSON encodes data values, not functions or every JavaScript type. Dates and domain types need explicit conventions.

Client-server architecture

P0 · Essential / Master · worked-example

The client presents interaction; the server validates requests and owns trusted business decisions.

Developer tools

P0 · Essential / Master · reference

Network, console, debugger, performance, and accessibility tools expose evidence about browser behavior.

Why Does It Exist?

A page that looks correct can still submit the wrong request. Understanding each boundary tells you whether to debug rendering, networking, server rules, or persistence.

How Does It Work?

Separate lookup, transport, protocol, and application behavior. A URL supplies a scheme, host, path, and possibly query. Inspect the request method, status, headers, and body before interpreting an error. A resolved fetch Promise says the transport produced a response; a 404 is still a response. A browser can reuse cached resources and existing connections, so do not assume every navigation repeats every step.

Examples

1. Trace the contract

```
const response = await fetch('/api/tasks', {headers:{Accept:'application/json'}});
if (!response.ok) throw new Error(`HTTP ${response.status}`);
if (!response.headers.get('content-type')?.includes('application/json')) {
  throw new Error('API returned a non-JSON representation');
}
const tasks = await response.json();
```

Expected behavior and runtime: In a served browser application, a successful JSON response becomes a value; a 404 or HTML fallback is reported as a failure. The example requires an actual /api/tasks endpoint.

2. Extend and stress the contract

Intermediate: deliberately request a missing API path and explain the representation. Advanced: compare a cold navigation with a repeat navigation and separate DNS, connection, and response caching. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Executable boundary: JSON boundaries and lost information

JSON is a data representation with a restricted value model. It does not preserve prototypes, undefined properties, circular references or arbitrary precision integers without an agreed encoding.

Concepts: JSON.

```
const encoded=JSON.stringify({date:new
  Date('2026-01-01T00:00:00Z'),missing:undefined,amount:12});
const decoded=JSON.parse(encoded);assert.equal(typeof decoded.date,'string');
assert.equal(Object.hasOwn(decoded,'missing'),false);
assert.throws(()=>JSON.stringify({amount:12n}),TypeError);
const
```

```
circular={};circular.self=circular;assert.throws(()=>JSON.stringify(circular),TypeError);
assert.throws(()=>JSON.parse('{bad json}'),SyntaxError);
assert.equal(JSON.parse('{"role":"owner"}').role,'owner'); // Parsing does not authorize
it.
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Dates become strings; undefined object properties disappear; BigInt/cycles throw. Well-formed JSON can still contain an unauthorized field.

Interview: Does successful JSON parsing establish trusted application data?

Answer: No. Check shape, field allowlists, types, size and domain authorization after parsing.

Change and verify: Validate a parsed expense with integer cents, an allowed category and a real calendar date. Reject an injected owner field.

Standalone executable.

Under the Hood

HTML parsing builds a document tree; CSS contributes style rules; layout computes geometry and painting produces pixels. Script and layout work can delay interaction. TLS authenticates the endpoint certificate, while application authorization still decides who may read a task.

Real-World Usage

Trace the MERN workspace login and task creation in Network tools. Compare a document request with a JSON request and identify where the session cookie travels.

Common Mistakes

- Treating HTTPS as proof that an application is trustworthy or authorized.
- Trying to fix a 500 response with CSS.
- Calling response.json() on every response without inspecting status or content type.

Best Practices

Trace the MERN workspace login and task creation in Network tools. Compare a document request with a JSON request and identify where the session cookie travels. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: identify scheme, host, path, method, status, and content type in three requests.
- Intermediate: deliberately request a missing API path and explain the representation.
- Advanced: compare a cold navigation with a repeat navigation and separate DNS, connection, and response caching.
- Challenge: trace one failed write and establish whether it reached the database.

Mini Project

Build a request diary containing five requests, their purpose, observed outcome, and one reproduced failure. Include screenshots with credentials redacted.

Project ladder and implementation status

Interview Questions

Beginner: Is JSON the same thing as a JavaScript object?

No. JSON is a text representation with a restricted value model; parsing creates JavaScript values.

Intermediate: Does fetch reject on 404?

No. Check response.ok or status. Transport failure and an HTTP error response are different.

Advanced: Why does a warm page load skip connection work?

The browser may reuse existing connections or cached resources. Measure the actual request timing.

Scenario: API returns HTML with status 200.

Inspect proxy and SPA fallback routing. APIs should return their defined representation and safe errors.

Debugging: Is the failure DNS or authorization?

DNS failure prevents locating a service; a 401 or 403 is an application response after transport succeeds.

Output-based: What does the worked example produce, and under which runtime?

In a served browser application, a successful JSON response becomes a value; a 404 or HTML fallback is reported as a failure. The example requires an actual /api/tasks endpoint.

Coding: What would you implement and verify next?

Build a request diary containing five requests, their purpose, observed outcome, and one reproduced failure. Include screenshots with credentials redacted.

Architecture and production: Which internal boundary needs evidence?

HTML parsing builds a document tree; CSS contributes style rules; layout computes geometry and painting produces pixels. Script and layout work can delay interaction. TLS authenticates the endpoint certificate, while application authorization still decides who may read a task.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Semantic HTML, forms, and accessible documents, REST contracts, GraphQL, and API evolution, Containers, CI/CD, and operating a web service

Next: Semantic HTML, forms, and accessible documents

Preserved lessons: 01-foundations/notes.md

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Semantic HTML, forms, and accessible documents

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: Internet, HTTP, and the browser

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Elements express document structure; attributes refine meaning and behavior. A valid document has a declared language and useful title.

Concept reference and priorities

Fundamentals

P0 · Essential / Master · reference

Elements express document structure; attributes refine meaning and behavior. A valid document has a declared language and useful title.

Semantic HTML

P0 · Essential / Master · worked-example

Use headings, landmarks, lists, links, and buttons for their meaning and built-in interaction.

Forms

P0 · Essential / Master · worked-example

Labels, names, input types, constraints, and submit behavior form a usable contract; the server validates again.

Tables

P1 · ☐ Highly Important · reference

Tables express relationships in rows and columns. Captions and header associations help users interpret the data.

Multimedia

P2 · Useful · reference

Images, audio, and video need alternatives appropriate to their purpose, including captions or transcripts when needed.

Accessibility

P0 · Essential / Master · worked-example

Accessibility includes keyboard interaction, names, focus, structure, contrast, and assistive-technology behavior.

SEO

P1 · Highly Important · reference

Descriptive titles, useful content, crawlable links, and meaningful structure help discovery; metadata is not a ranking guarantee.

HTML APIs

P2 · Useful · reference

Browser APIs such as dialog, history, and storage have specific lifecycle and permission contracts beyond markup.

Modern HTML

P1 · Highly Important · reference

Native dialog, disclosure, responsive images, and newer platform features can reduce custom code; check support for your audience.

Why Does It Exist?

A styled clickable div requires you to rebuild behavior that a button already has. Semantic markup keeps meaning and interaction aligned as visual design changes.

How Does It Work?

A form is a set of named controls submitted through a defined action or handled by code. The label identifies purpose, name identifies submitted data, and id establishes document relationships. A placeholder disappears while typing and is not a reliable label. Choose a link for navigation and a button for an action. Keep source order meaningful before applying visual layout.

Examples

1. Trace the contract

```
<html lang="en">
<head><title>Reading list</title></head>
<body><main>
<h1>Add a reading-list item</h1>
<form action="/api/bookmarks" method="post">
<label for="title">Title</label>
<input id="title" name="title" required maxlength="120">
<button type="submit">Add item</button>
</form>
</main></body></html>
```

Expected behavior and runtime: Typing and pressing Enter submits the named title field. This markup assumes a form-capable endpoint; the existing JSON API needs a JavaScript submit handler instead.

2. Extend and stress the contract

Intermediate: add an error linked with aria-describedby and preserve the input. Advanced: add captions and responsive sources to educational media. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

The browser derives an accessibility tree from elements, attributes, and state. ARIA can refine this tree, but does not add keyboard behavior to arbitrary elements. Native controls also participate in focus and form submission.

Real-World Usage

Use the UI lab to inspect an accordion and dialog. Build a table of expenses with a caption, row/column headers, and a small-screen scrolling container.

Common Mistakes

- Using headings only for their default font sizes.
- Removing visible focus outlines without an equivalent replacement.
- Trusting required or pattern attributes as a server security boundary.

Best Practices

Use the UI lab to inspect an accordion and dialog. Build a table of expenses with a caption, row/column headers, and a small-screen scrolling container. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: navigate the form using only the keyboard.
- Intermediate: add an error linked with aria-describedby and preserve the input.
- Advanced: add captions and responsive sources to educational media.
- Challenge: test at 200 percent zoom and with a screen reader; record specific obstacles.

Mini Project

Build a semantic profile and contact form. Acceptance: useful title and language, ordered headings, labeled controls, keyboard submission, and errors associated with the correct input.

Project ladder and implementation status

Interview Questions

Beginner: When is a button better than a link?

For actions; a link navigates to a destination.

Intermediate: What does name do in a form?

It identifies a successful control in submitted form data; id serves document relationships.

Advanced: Why does ARIA not make a div a complete button?

ARIA changes exposed semantics but keyboard, focus, and activation still need implementation.

Scenario: A modal closes but focus disappears.

Remember the invoking element and return focus if it remains available; use a native dialog when suitable.

Debugging: A label focuses the wrong input.

Check its for value and the uniqueness of the matching input id.

Output-based: What does the worked example produce, and under which runtime?

Typing and pressing Enter submits the named title field. This markup assumes a form-capable endpoint; the existing JSON API needs a JavaScript submit handler instead.

Coding: What would you implement and verify next?

Build a semantic profile and contact form. Acceptance: useful title and language, ordered headings, labeled controls, keyboard submission, and errors associated with the correct input.

Architecture and production: Which internal boundary needs evidence?

The browser derives an accessibility tree from elements, attributes, and state. ARIA can refine this tree, but does not add keyboard behavior to arbitrary elements. Native controls also participate in focus and form submission.

[Difficulty-based interview bank](#) · [Scenario practice](#) · [Debugging library](#)

Related Concepts

CSS layout, cascade, and responsive design, JavaScript values, scope, functions, and collections, React identity, state, effects, and resilient interfaces

Next: [CSS layout, cascade, and responsive design](#)

Preserved lessons: [notes/html.md](#), [02-html/notes.md](#)

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the [verified media register](#). A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

[Local project ladder](#) · [Timed exercises](#)

Further Reading

[Curated external register](#) — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

CSS layout, cascade, and responsive design

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: Semantic HTML, forms, and accessible documents

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

CSS maps selectors to declarations; browsers resolve competing declarations and compute values for elements.

Concept reference and priorities

Fundamentals

P0 · Essential / Master · reference

CSS maps selectors to declarations; browsers resolve competing declarations and compute values for elements.

Box model

P0 · Essential / Master · worked-example

Content, padding, border, and margin determine geometry. border-box includes padding and borders in the declared width.

Selectors

P0 · Essential / Master · reference

Selectors match document structure and states; prefer intentional classes over tightly coupled ancestor chains.

Cascade

P0 · Essential / Master · reference

Origin, importance, layers, specificity, scope proximity, and order resolve competing declarations.

Specificity

P0 · Essential / Master · reference

Specificity compares selector weight within the relevant cascade stage; it is not the first rule in every conflict.

Flexbox

P0 · Essential / Master · reference

Flexbox distributes items along an axis and manages wrapping, alignment, and available space.

Grid

P0 · Essential / Master · worked-example

Grid defines tracks in two dimensions and can coordinate row and column placement.

Responsive design

P0 · Essential / Master · worked-example

A responsive layout adapts to available space, input capabilities, text size, and content length.

Media queries

P1 · ☐ Highly Important · reference

Media queries condition rules on viewport or device characteristics; choose breakpoints from content needs.

Animations

P2 · Useful · reference

Keyframe animations interpolate properties over time; honor reduced-motion preferences and avoid unnecessary layout work.

Transitions

P2 · Useful · reference

Transitions interpolate a changed property over a duration; they need distinct before and after values.

Positioning

P1 · ☐ Highly Important · reference

Static, relative, absolute, fixed, and sticky positioning use different containing-block and scroll relationships.

Variables

P1 · ☐ Highly Important · worked-example

Custom properties cascade and can be resolved at use time; they are useful for design tokens and themes.

Accessibility

P0 · Essential / Master · worked-example

Preserve readable contrast, focus, zoom, reduced motion, and logical source order.

Modern CSS

P2 · Useful · reference

Container queries, cascade layers, subgrid, logical properties, and modern selectors solve specific layout problems; inspect compatibility.

CSS architecture

P1 · ☐ Highly Important · reference

Small reusable patterns, consistent naming, and documented tokens reduce unintended cross-component effects.

Why Does It Exist?

Layout failures often come from content constraints, not a missing breakpoint. CSS offers different tools for distribution, track alignment, overlap, and local component adaptation.

How Does It Work?

Start in normal flow, choose a layout model, and identify intrinsic size constraints. In a grid, a long word can impose a minimum track width; `minmax(0, 1fr)` lets a flexible track shrink when overflow handling is defined. In flex layouts, `min-width: 0` often permits a child to shrink. Inspect which cascade rule wins before raising specificity. A large z-index cannot escape an ancestor stacking context.

Examples

1. Trace the contract

```
:root { --space: 1rem; --accent: #214ea2; }
* { box-sizing: border-box; }
.cards { display: grid; gap: var(--space);
grid-template-columns: repeat(auto-fit, minmax(min(100%, 16rem), 1fr)); }
.card { min-width: 0; overflow-wrap: anywhere; }
button:focus-visible { outline: 3px solid var(--accent); outline-offset: 3px; }
@media (prefers-reduced-motion: reduce) {
  .animated { animation: none; transition: none; }
}
```

Expected behavior and runtime: Cards wrap into available columns and long text can wrap inside a shrinking card. Inspect 320px and desktop layouts with real content; this is a stylesheet fragment for a page containing these classes.

2. Extend and stress the contract

Intermediate: fix a long URL that expands a grid. Advanced: debug a modal behind a transformed ancestor using stacking-context evidence. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Browsers compute style, layout, paint, and compositing work. Some visual changes require geometry to be recalculated. Transform and opacity often avoid layout, but layer promotion and rendering costs still need measurement.

Real-World Usage

Use these rules in a dashboard with unequal titles and amounts. Give tabular data an intentional overflow container rather than hiding part of a row.

Common Mistakes

- Using `overflow:hidden` to conceal an inaccessible control.
- Adding `!important` before inspecting competing layers and selectors.
- Designing only for one viewport or short English labels.

Best Practices

Use these rules in a dashboard with unequal titles and amounts. Give tabular data an intentional overflow container rather than hiding part of a row. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: draw and measure one element with both box-sizing values.
- Intermediate: fix a long URL that expands a grid.
- Advanced: debug a modal behind a transformed ancestor using stacking-context evidence.
- Challenge: produce a readable layout at 320px, 200 percent zoom, and reduced motion.

Mini Project

Build a responsive reading-list dashboard with cards, a filter toolbar, visible focus, and a data table. Record why each section uses Grid, Flexbox, or normal flow.

Project ladder and implementation status

Interview Questions

Beginner: What does border-box change?

The declared width includes content, padding, and border; margin remains outside.

Intermediate: Why can a less specific rule win?

Earlier cascade stages such as importance and layer order can take precedence over specificity.

Advanced: Why is z-index ineffective?

The element may participate in an ancestor stacking context that is ordered behind another context.

Scenario: A card expands the mobile page.

Inspect intrinsic minimum size and long content; allow shrinking and intentional wrapping.

Debugging: Sticky positioning does not appear to stick.

Check its inset, containing block, available scroll distance, and ancestor overflow behavior.

Output-based: What does the worked example produce, and under which runtime?

Cards wrap into available columns and long text can wrap inside a shrinking card. Inspect 320px and desktop layouts with real content; this is a stylesheet fragment for a page containing these classes.

Coding: What would you implement and verify next?

Build a responsive reading-list dashboard with cards, a filter toolbar, visible focus, and a data table. Record why each section uses Grid, Flexbox, or normal flow.

Architecture and production: Which internal boundary needs evidence?

Browsers compute style, layout, paint, and compositing work. Some visual changes require geometry to be recalculated. Transform and opacity often avoid layout, but layer promotion and rendering costs still need measurement.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

React identity, state, effects, and resilient interfaces, Internet, HTTP, and the browser

Next: React identity, state, effects, and resilient interfaces

Preserved lessons: [notes/css.md](#), [03-css/notes.md](#)

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

JavaScript values, scope, functions, and collections

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: Internet, HTTP, and the browser

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

JavaScript evaluates expressions and statements using a language-defined value model and host-provided APIs.

Concept reference and priorities

Fundamentals

P0 · Essential / Master · worked-example

JavaScript evaluates expressions and statements using a language-defined value model and host-provided APIs.

Variables

P0 · Essential / Master · worked-example

Bindings associate names with values; const prevents reassignment of a binding, not mutation of its object.

Data types

P0 · Essential / Master · worked-example

Primitives and objects have different identity behavior. undefined, null, numbers, strings, booleans, bigint, and symbols need explicit handling.

Operators

P0 · Essential / Master · worked-example

Operators combine or compare values; coercion, short-circuiting, and nullish behavior affect results.

Control flow

P0 · Essential / Master · worked-example

Conditions, loops, return, break, and continue select which work runs and when iteration ends.

Functions

P0 · Essential / Master · worked-example

Functions define callable behavior, parameters, return values, and closures over lexical bindings.

Scope

P0 · Essential / Master · worked-example

Lexical scope resolves names from where code is written, not from whichever function called it.

Closures

P0 · Essential / Master · worked-example

A function retains access to its lexical environment, including bindings whose values can change.

Hoisting

P1 · ☐ Highly Important · worked-example

Declarations are instantiated before evaluation, but initialization timing differs; let and const have a temporal dead zone.

this

P1 · ☐ Highly Important · worked-example

Ordinary function this depends on invocation; arrows capture the surrounding this and cannot be rebound with call.

Objects

P0 · Essential / Master · worked-example

Objects group properties and have identity; spreading creates a shallow copy of own enumerable properties.

Arrays

P0 · Essential / Master · worked-example

Arrays hold indexed collections; map transforms, filter selects, and reduce accumulates values.

Destructuring

P1 · ☐ Highly Important · worked-example

Destructuring binds values from structured data; defaults apply to undefined rather than every falsy value.

Spread/rest

P1 · ☐ Highly Important · worked-example

Spread expands values; rest collects remaining arguments or properties. Object spread does not deeply clone nested objects.

Prototypes

P2 · Useful · worked-example

Property lookup can follow a prototype chain after own properties are checked.

Classes

P2 · Useful · reference

Classes provide syntax around constructors and prototype-based methods, with additional class-specific semantics.

Inheritance

P2 · Useful · worked-example

Inheritance reuses behavior through type or prototype relationships; composition can avoid rigid hierarchies.

Modules

P0 · Essential / Master · reference

Modules make dependencies and exports explicit; ESM and CommonJS have different loading and interoperability rules.

Error handling

P0 · Essential / Master · worked-example

Throw, try/catch, and Promise rejection represent failures; handle errors at a boundary that can act.

Memory

P1 · ☐ Highly Important · worked-example

Retained references determine whether objects remain reachable; caches, listeners, and closures can retain large graphs.

Garbage collection

P2 · Useful · reference

The engine reclaims unreachable objects; collection timing is not a portable application guarantee.

Iterators

P2 · Useful · worked-example

An iterator returns successive value/done results; iterable protocols allow for-of and spread to consume sequences.

Generators

P3 · Advanced / Specialized · worked-example

Generators suspend and resume function execution while yielding values, providing convenient iterator state.

Symbols

P3 · Advanced / Specialized · reference

Symbols create distinct property keys and participate in protocols such as iteration.

Proxy

P3 · Advanced / Specialized · reference

A Proxy intercepts operations on a target through traps; language invariants constrain permitted behavior.

Reflect

P3 · Advanced / Specialized · reference

Reflect exposes operations corresponding to object internal behavior and can preserve forwarding semantics in traps.

Typed arrays

P3 · Advanced / Specialized · reference

Typed arrays view fixed-format binary values in buffers; byte layout and bounds matter.

Internationalization

P1 · ☐ Highly Important · reference

Intl formats numbers, dates, and text according to locale conventions; formatting is separate from domain storage.

Modern ECMAScript

P2 · Useful · reference

Language additions vary by runtime support. Check the target runtime rather than assuming a proposal is standardized.

TC39 proposals

P4 · Reference / Experimental · reference

Proposals have explicit stages and can change or stop progressing; inspect proposal status and runtime support before relying on experimental behavior.

Performance

P1 · ☐ Highly Important · reference

Measure workload, allocation, and blocking time before replacing readable code with an optimization.

Security

P0 · Essential / Master · reference

Treat untrusted values as data; validate at boundaries and avoid executing input or inserting it as HTML.

Why Does It Exist?

Many application bugs are mistaken assumptions about identity, coercion, or captured values. A precise value model also makes React updates and backend validation easier to explain.

How Does It Work?

Start with a small input, trace each binding, and state the output. Object copies and immutable transitions are separate decisions: a new outer object can still share a nested array. Use `Number.isFinite` and explicit parsing rules for numeric inputs. Use a `Map` when key identity matters and plain records when the domain is fixed. Design functions with a documented input, result, and failure contract.

Examples

1. Trace the contract

```
const original = {tags:['mern']};
const copy = {...original};
copy.tags.push('interview');
console.log(original.tags.length); // 2: nested array is shared
```

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const next = makeCounter();
console.log(next(), next()); // 1 2
```

Expected behavior and runtime: The log values are 2, then 1 and 2. Replace tags with a new array to avoid mutating the original; each `makeCounter` call creates a distinct environment.

2. Extend and stress the contract

Intermediate: predict shallow-copy and equality outputs before running them. Advanced: implement a generator over a bounded tree and show early termination. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Executable boundary: Scope and closure capture

A closure retains access to a binding, not a photograph of its value. A `let` binding inside a `for` loop is recreated for each iteration; a `var` binding is shared by callbacks.

Concepts: Scope, Closures, Control flow.

```
const shared=[]; for(var n=0;n<3;n++)shared.push(()=>n);
const isolated=[]; for(let i=0;i<3;i++)isolated.push(()=>i);
assert.deepEqual(shared.map(fn=>fn()),[3,3,3]);
assert.deepEqual(isolated.map(fn=>fn()),[0,1,2]);
function counter(){let value=0;return()=>++value;}
const first=counter(),second=counter();
assert.deepEqual([first(),first(),second()], [1,2,1]);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Shared callbacks return 3; per-iteration callbacks return 0, 1, 2. Separately created counters have separate lexical environments.

Interview: Why does changing var to let fix the callback example?

Answer: The loop supplies separate lexical bindings. Scheduling is irrelevant here: every callback is invoked after the loop.

Change and verify: Return read and increment methods from counter. Verify they share one private binding, while a second counter stays independent.

Standalone executable.

Executable boundary: Hoisting and the temporal dead zone

Creation of a lexical binding and initialization of its value are separate. Reading a lexical binding before initialization throws; a function declaration is available when its enclosing scope starts executing.

Concepts: Hoisting, Variables.

```
assert.equal(declared(),7);
function declared(){return 7;}
assert.throws(()=>{const read=()=>later;read();let later=9;},ReferenceError);
assert.throws(()=>{const read=()=>typeof later;read();let later=9;},ReferenceError);
assert.equal(typeof genuinelyMissing,'undefined');
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. The declaration returns 7. Even typeof throws for an uninitialized lexical binding; typeof an undeclared identifier returns undefined.

Interview: Is let not hoisted?

Answer: Its binding exists before initialization and shadows outer bindings, but cannot be read during that interval. Calling this simply unhoisted hides the temporal dead zone.

Change and verify: Add an outer later value and show that the inner lexical binding still shadows it before initialization.

Standalone executable.

Executable boundary: Method receiver and explicit binding

An ordinary function receives this according to its invocation. Extracting a method loses the receiver. An arrow retains lexical this and cannot be rebound with call.

Concepts: this, Functions, Objects.

```
const account={value:8,read(){return this.value;}};
assert.equal(account.read(),8);
const detached=account.read;
assert.throws(()=>detached(),TypeError); // ESM is strict.
assert.equal(detached.call({value:5}),5);
const bound=detached.bind(account);
assert.equal(bound.call({value:99}),8);
function createArrow(){return()=>this.value;}
const arrow=createArrow.call(account);
assert.equal(arrow.call({value:100}),8);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Method call 8, explicit receiver 5, bound receiver 8, and lexical arrow receiver 8. Detached invocation throws in this ESM runtime.

Interview: Does an arrow function have its own this?

Answer: No. It resolves this through the enclosing lexical scope. A method shorthand is an ordinary function and follows invocation rules.

Change and verify: Pass the method through a callback API and fix receiver loss with binding or a closure; explain which value is retained.

Standalone executable.

Executable boundary: Prototypes, inheritance and property descriptors

Lookup walks the prototype chain, while ownership and enumeration describe different properties. A non-writable own property rejects assignment in strict code.

Concepts: Prototypes, Inheritance, Objects.

```
const parent={kind:'record'};
const child=Object.create(parent);child.title='Draft';
assert.equal(child.kind,'record');assert.equal(Object.hasOwn(child,'kind'),false);
assert.deepEqual(Object.keys(child),['title']);
Object.defineProperty(child,'id',{value:42,writable:false,enumerable:false});
assert.throws(()=>{child.id=99;},TypeError);
assert.equal(child.id,42);assert.equal(JSON.stringify(child),'{"title":"Draft"}');
child.kind='article';assert.equal(parent.kind,'record');
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Inherited kind is readable but is not an own key. Hidden id remains 42 and is omitted from JSON. Shadowing kind leaves the parent unchanged.

Interview: Does Object.keys include inherited properties?

Answer: No; it returns enumerable own string keys. A for-in loop can visit inherited enumerable keys and therefore needs a deliberate ownership policy.

Change and verify: Compare Object.keys, Reflect.ownKeys and for-in after adding a Symbol key and an inherited enumerable field.

Standalone executable.

Executable boundary: Copy depth and array transformations

Spread copies one level. Nested objects remain shared references. Non-mutating array operations can preserve input order/identity, but copying the outer array does not clone its elements.

Concepts: Arrays, Spread/rest, Destructuring, Memory.

```
const input=[{id:1,meta:{done:false}},{id:2,meta:{done:true}}];
const shallow=[...input];shallow[0].meta.done=true;
assert.equal(input[0].meta.done,true);
const separate=structuredClone(input);separate[0].meta.done=false;
assert.equal(input[0].meta.done,true);
const ids=input.map(({id})=>id);assert.deepEqual(ids,[1,2]);
assert.deepEqual(ids.toSorted((a,b)=>b-a),[2,1]);assert.deepEqual(ids,[1,2]);
assert.equal(input.filter(row=>row.meta.done).length,2);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Shallow copy shares nested meta; structuredClone separates these cloneable records. toSorted returns a new array and leaves ids unchanged.

Interview: Is [...rows] enough for an immutable nested update?

Answer: No. Copy each object along the changed path, or use a suitable immutable update tool. Preserve the identity of unaffected rows.

Change and verify: Implement a nested completion toggle with map and nested spread. Verify the original record stays unchanged and unaffected rows keep identity.

Standalone executable.

Executable boundary: Coercion, equality and numeric limits

Operators define their own conversion rules. Explicit input conversion and finite/safe-integer checks make domain boundaries clearer than relying on truthiness or loose equality.

Concepts: Data types, Operators, Fundamentals.

```
assert.equal('2'+1, '21');assert.equal('2'-1,1);
assert.equal(Number(''),0);assert.equal(Boolean('false'),true);
assert.equal(0===' ',true);assert.equal(0=== '',false);
assert.equal(NaN===NaN,false);assert.equal(Object.is(NaN,NaN),true);
assert.equal(0===-0,true);assert.equal(Object.is(0,-0),false);
assert.equal(Number.isSafeInteger(9007199254740992),false);
assert.equal(0.1+0.2===0.3,false);
assert.equal(10+20,30); // Integer cents for a money domain.
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. String addition concatenates, subtraction converts, empty strings convert to zero, and nonempty false text is truthy. Integer cents avoid this simple fractional sum error.

Interview: Can Number(input) alone validate an amount?

Answer: No. It accepts blank input as zero, and it can produce nonfinite or unsafe values. Validate syntax and domain range as well as conversion.

Change and verify: Reject blank, negative, exponential and more-than-two-decimal currency strings. Reuse the exact-cent money parser from the MERN workspace.

Standalone executable.

Executable boundary: Iterators and generator cleanup

A generator suspends its execution and implements the iterator protocol. Early termination of a for-of loop calls return when supplied, allowing a finally block to release resources.

Concepts: Iterators, Generators.

```
let closed=false;
function* pages(){try{yield 1;yield 2;}finally{closed=true;}}
for(const page of pages()){assert.equal(page,1);break;}
assert.equal(closed,true);
const iterator=pages();assert.deepEqual(iterator.next(),{value:1,done:false});
assert.deepEqual(iterator.next(),{value:2,done:false});
assert.deepEqual(iterator.next(),{value:undefined,done:true});
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Breaking the loop executes finally. Manual next calls reveal value/done pairs and completion.

Interview: Does a generator run its body immediately when called?

Answer: No; calling it returns an iterator. Execution begins when iteration requests a value, and resumes after each yield.

Change and verify: Replace the fake pages with an async generator. Prove that breaking the consumer closes the iterator without requesting another page.

Standalone executable.

Executable boundary: Error propagation and finally

An exception transfers control to the nearest compatible catch/finally path. Returning from finally overrides a pending result or exception, which can hide a failed operation.

Concepts: Error handling.

```
function hidden(){try{throw new Error('lost');}finally{return 'masked';}}
assert.equal(hidden(),'masked');
function visible(){try{throw new Error('visible');}finally{/* release resource; no return
*}}
assert.throws(visible,/visible/);
const cause=new Error('database offline');const wrapper=new Error('Save failed',{cause});
assert.equal(wrapper.cause,cause);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. The first failure is masked; the second propagates. An Error cause preserves a lower-level failure without copying private details into a public response.

Interview: Should an API return the whole caught error?

Answer: No. Translate to a stable safe error contract, record permitted diagnostic context privately, and preserve useful causes internally.

Change and verify: Add cleanup to a rejected async save. Verify cleanup runs and the caller still observes rejection; avoid returning a success from finally.

Standalone executable.

Under the Hood

A closure retains an environment, not an automatic immutable copy of every captured value. A normal method can lose its receiver when passed as a bare callback. Prototype lookup does not imply that arbitrary input should be merged into privileged configuration.

Real-World Usage

Use pure collection transformations to derive a UI list. Use explicit allowlists to build an API update rather than spreading request.body. Compare the toolkit emitter and LRU choices with plain objects.

Common Mistakes

- Using Boolean(value) to parse the string false.
- Treating const or object spread as deep immutability.
- Using loose coercion for dates, money, or authorization decisions.

Best Practices

Use pure collection transformations to derive a UI list. Use explicit allowlists to build an API update rather than spreading request.body. Compare the toolkit emitter and LRU choices with plain objects. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: write a loop that includes zero and explain its termination.
- Intermediate: predict shallow-copy and equality outputs before running them.
- Advanced: implement a generator over a bounded tree and show early termination.
- Challenge: implement an LRU and explain eviction with stored undefined values.

Mini Project

Build a pure quiz scoring model with explicit invalid-answer behavior, then connect it to a browser form. Add output examples for blank input, duplicate answers, and a changed question order.

Project ladder and implementation status

Interview Questions

Beginner: Does const freeze an object?

No. It prevents rebinding the variable; object properties can still change.

Intermediate: Why does a copied object mutate the original array?

The copy preserves the nested reference. Replace or clone at the depth required by the domain.

Advanced: How does this differ in an arrow?

The arrow captures lexical this; an ordinary function derives this from invocation.

Scenario: Memory grows after every page visit.

Compare heap snapshots and retaining paths for unremoved listeners, unbounded caches, and captured objects.

Debugging: A default value replaces a valid zero.

Use a nullish check when only null and undefined are missing; || also treats zero and an empty string as falsy.

Output-based: What does the worked example produce, and under which runtime?

The log values are 2, then 1 and 2. Replace tags with a new array to avoid mutating the original; each makeCounter call creates a distinct environment.

Coding: What would you implement and verify next?

Build a pure quiz scoring model with explicit invalid-answer behavior, then connect it to a browser form. Add output examples for blank input, duplicate answers, and a changed question order.

Architecture and production: Which internal boundary needs evidence?

A closure retains an environment, not an automatic immutable copy of every captured value. A normal method can lose its receiver when passed as a bare callback. Prototype lookup does not imply that arbitrary input should be merged into privileged configuration.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Promises, event loops, and bounded concurrency, DOM, events, and browser APIs, TypeScript and validated application boundaries, Node.js runtime, resources, and asynchronous services

Next: Promises, event loops, and bounded concurrency

Preserved lessons: notes/javascript.md, 04-javascript/notes.md

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

TC39 proposal register — checked 2026-10-08; proposal status is not a guarantee of shipped runtime support.

Promises, event loops, and bounded concurrency

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: JavaScript values, scope, functions, and collections

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

A callback is a function another operation invokes; it can run synchronously or asynchronously according to that contract.

Concept reference and priorities

Callbacks

P0 · Essential / Master · worked-example

A callback is a function another operation invokes; it can run synchronously or asynchronously according to that contract.

Promises

P0 · Essential / Master · worked-example

A Promise represents an eventual outcome. Chaining transforms outcomes and propagates failures.

async/await

P0 · Essential / Master · worked-example

An async function returns a Promise; await suspends that function until the awaited outcome is available.

Event loop

P0 · Essential / Master · worked-example

A host coordinates execution and queued work. Browser rendering and Node phases have distinct scheduling models.

Microtasks

P1 · ☐ Highly Important · worked-example

Promise reactions and queueMicrotask callbacks run at host-defined checkpoints after the current work.

Macrotasks

P1 · ☐ Highly Important · worked-example

Task is the browser-standard term; timers and other task sources schedule later work with no exact execution-time guarantee.

Fetch

P0 · Essential / Master · worked-example

Fetch produces an HTTP response or transport failure; the application still checks status and parses the expected representation.

Async programming

P0 · Essential / Master · worked-example

Design ordering, cancellation, concurrency limits, timeouts, and partial-failure behavior explicitly.

Why Does It Exist?

Parallel work can reduce waiting but overwhelm a database or remote service. Waiting for every request is different from limiting how many requests start at once.

How Does It Work?

A Promise executor runs immediately; its reactions run later. Promise.all waits on already-started inputs and rejects on the first rejection without canceling peers. allSettled retains each outcome. await does not make synchronous CPU work leave the current thread. For cancellation, pass a supported signal and define what happens to already-committed server work.

Examples

1. Trace the contract

```
console.log('A');
Promise.resolve().then(() => console.log('B'));
queueMicrotask(() => console.log('C'));
setTimeout(() => console.log('D'), 0);
console.log('E');
```

Expected behavior and runtime: For this ordinary top-level browser script: A, E, B, C, D. Do not extrapolate this small example to every Node phase, nested task source, or rendering opportunity.

2. Extend and stress the contract

Intermediate: compare fail-fast and all-settled batch contracts. Advanced: test the maximum active count with an injected deferred task. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Executable boundary: Promise settlement and concurrent outcomes

Promise.all is fail-fast at its observation boundary; it does not cancel work already started. allSettled collects independent successes and failures. Await catches rejection where it is awaited.

Concepts: Promises, async/await, Async programming.

```
let completed=0;
const later=new Promise(resolve=>setTimeout(()=>{completed++;resolve('saved');},5));
await assert.rejects(Promise.all([Promise.reject(new Error('failure')),later]),/failure/);
assert.equal(await later,'saved');assert.equal(completed,1);
const outcomes=await Promise.allSettled([Promise.resolve(3),Promise.reject('offline')]);
assert.deepEqual(outcomes.map(row=>row.status),['fulfilled','rejected']);
assert.equal(outcomes[0].value,3);assert.equal(outcomes[1].reason,'offline');
assert.equal(await Promise.resolve(1).then(n=>n+1).finally(()=>99),2);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. all rejects, yet the already-started later operation completes. allSettled returns both outcomes. A normally returning finally preserves the previous result.

Interview: Is Promise.all a concurrency limiter?

Answer: No. If requests are created before it is called, those requests have already started. Limit admission with a bounded worker pool.

Change and verify: Use the tested promise pool in js-toolkit with a cap of two; instrument active work and prove its maximum does not exceed the cap.

Standalone executable.

Executable boundary: Tasks and microtasks in an ESM trace

Synchronous work finishes before queued Promise callbacks and queueMicrotask callbacks run. A later timer task sees the microtask queue drained. This example deliberately avoids ambiguous sibling timer/immediate ordering.

Concepts: Event loop, Microtasks, Macrotasks, Callbacks.

```
const order=[];
order.push('sync');
queueMicrotask(()=>order.push('microtask'));
Promise.resolve().then(()=>{order.push('promise');queueMicrotask(()=>order.push('nested'))});
});
await new Promise(resolve=>setTimeout(()=>{order.push('timer');resolve();},0));
assert.deepEqual(order,['sync','microtask','promise','nested','timer']);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. sync, microtask, promise, nested, timer under Node 24 ESM. Promise jobs and explicit microtasks run in enqueue order in this trace.

Interview: Why can recursively queued microtasks freeze a browser?

Answer: They can prevent the queue from draining and delay rendering and later tasks. Break large work into bounded chunks or move suitable CPU work to a worker.

Change and verify: Add a microtask inside the first microtask. Predict the full ordering before running; do not infer an ordering between all host scheduling APIs.

Standalone executable.

Executable boundary: AbortSignal and cooperative cancellation

Cancellation is a signal observed by an operation. Promise.race by itself only chooses an outcome; it does not stop the losing operation. Fetch accepts a signal, while your own functions must implement cleanup.

Concepts: Fetch, Async programming.

```
function wait(signal){return new Promise((resolve,reject)=>{
if(signal.aborted){reject(signal.reason);return;}
const abort=()=>{clearTimeout(timer);reject(signal.reason);};
const
  timer=setTimeout(()=>{signal.removeEventListener('abort',abort);resolve('done');},100);
signal.addEventListener('abort',abort,{once:true});
});}
const controller=new AbortController(),work=wait(controller.signal);
controller.abort(new Error('superseded'));await assert.rejects(work,/superseded/);
await assert.rejects(wait(controller.signal),/superseded/);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Both in-flight and already-aborted requests reject with superseded. The timer is canceled and the registered listener is one-shot.

Interview: Is aborting enough to protect a search UI from stale results?

Answer: No. The operation might already have completed or might ignore the signal. Also compare a request generation before committing its result; the weather/browser fixtures demonstrate that boundary.

Change and verify: Replace the timer with a controllable fake provider. Resolve the old request after the new one and prove the old result never replaces the current view.

Standalone executable.

Under the Hood

The call stack must finish before these queued reactions execute. A chain that continually adds microtasks can postpone other work. Timer delay is a lower-bound scheduling condition, not a promise of an exact deadline.

Real-World Usage

Use the tested promisePool to process a batch with a concurrency cap and input-order results. Use an AbortController and a request generation to prevent older search results from replacing newer UI state.

Common Mistakes

- Starting every fetch before passing the promises to a purported concurrency limiter.
- Assuming Promise.all cancels the losing requests.
- Blocking an API with expensive synchronous work inside an async function.

Best Practices

Use the tested promisePool to process a batch with a concurrency cap and input-order results. Use an AbortController and a request generation to prevent older search results from replacing newer UI state. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: predict the five log lines before running the example.
- Intermediate: compare fail-fast and all-settled batch contracts.
- Advanced: test the maximum active count with an injected deferred task.
- Challenge: design timeout and cancellation outcomes for already-running tasks without losing result order.

Mini Project

Extend the toolkit promise pool with a documented abort policy. Acceptance: no new tasks after abort, existing tasks accounted for, bounded active work, and stable result ordering.

Project ladder and implementation status

Interview Questions

Beginner: What does an async function return?

A Promise, including when the function returns a plain value.

Intermediate: Does await parallelize a loop?

No. Awaiting each iteration usually serializes starts; choose controlled concurrency explicitly.

Advanced: Can Promise.allSettled bound load?

No. It collects outcomes from inputs; a worker or scheduler must limit when tasks start.

Scenario: An old search response wins.

Associate data with the current query and discard stale results; cancellation alone is not an ordering proof.

Debugging: A timer runs late.

Inspect blocking JavaScript, microtask starvation, and host scheduling; a timer is not an exact real-time guarantee.

Output-based: What does the worked example produce, and under which runtime?

For this ordinary top-level browser script: A, E, B, C, D. Do not extrapolate this small example to every Node phase, nested task source, or rendering opportunity.

Coding: What would you implement and verify next?

Extend the toolkit promise pool with a documented abort policy. Acceptance: no new tasks after abort, existing tasks accounted for, bounded active work, and stable result ordering.

Architecture and production: Which internal boundary needs evidence?

The call stack must finish before these queued reactions execute. A chain that continually adds microtasks can postpone other work. Timer delay is a lower-bound scheduling condition, not a promise of an exact deadline.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

DOM, events, and browser APIs, Node.js runtime, resources, and asynchronous services, Behavioral testing across units, databases, and browsers

Next: DOM, events, and browser APIs

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

DOM, events, and browser APIs

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: Semantic HTML, forms, and accessible documents, JavaScript values, scope, functions, and collections, Promises, event loops, and bounded concurrency

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

The DOM exposes a document tree as objects that scripts can inspect and update.

Concept reference and priorities

DOM

P0 · Essential / Master · worked-example

The DOM exposes a document tree as objects that scripts can inspect and update.

Events

P0 · Essential / Master · worked-example

Events describe interaction or state changes; propagation and default actions follow distinct rules.

Browser APIs

P1 · ☐ Highly Important · reference

Storage, history, observers, workers, and permissions have host-specific contracts and cleanup requirements.

Forms and events

P0 · Essential / Master · reference

A submit event captures keyboard and button submission; form controls should retain accessible names and feedback.

Why Does It Exist?

Event-driven interaction lets the page change without reconstructing all document state. Correct event ownership avoids duplicate listeners, accidental navigation, and unsafe content insertion.

How Does It Work?

Separate propagation from default behavior. `preventDefault` affects the default action; `stopPropagation` affects propagation. Event delegation listens on a stable ancestor and identifies a meaningful target, including when the user clicks a nested child. Update user-supplied text with `textContent`. Keep storage as an optional convenience and handle quota, unavailable storage, and malformed values.

Examples

1. Trace the contract

```
const list = document.querySelector('#items');
list.addEventListener('click', event => {
  const button = event.target instanceof Element
    ? event.target.closest('button[data-id]') : null;
  if (!button || !list.contains(button)) return;
  console.log('Requested item', button.dataset.id);
});
```

Expected behavior and runtime: A click on text or an icon inside a matching button identifies that button. The containing list must exist before registering the listener; ids remain untrusted inputs at the server boundary.

2. Extend and stress the contract

Intermediate: add a row without registering a new row listener. Advanced: handle a storage write failure while preserving export. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Events can pass through capture, target, and bubble phases. Shadow DOM can retarget events. Registering listeners repeatedly can retain state and cause duplicate work. Most DOM updates do not automatically preserve focus after replacing an element.

Real-World Usage

Extend the UI lab to handle dynamically inserted todos, focus a useful control after deletion, and export progress without depending on localStorage always succeeding.

Common Mistakes

- Using innerHTML with untrusted titles.
- Listening only for button clicks and missing keyboard form submission.
- Persisting session secrets in a progress store.

Best Practices

Extend the UI lab to handle dynamically inserted todos, focus a useful control after deletion, and export progress without depending on localStorage always succeeding. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: inspect target and currentTarget on a nested button click.
- Intermediate: add a row without registering a new row listener.
- Advanced: handle a storage write failure while preserving export.
- Challenge: replace a list while preserving a sensible keyboard focus target.

Mini Project

Build a local notes app with a plain-text editor, explicit save state, JSON export, and safe handling of corrupted saved data. Do not present browser storage as a multi-device database.

Project ladder and implementation status

Interview Questions

Beginner: How do target and currentTarget differ?

Target identifies the dispatch target; currentTarget is the element whose listener is currently running.

Intermediate: Why use delegation?

A stable ancestor can handle matching descendants added later without per-row listeners.

Advanced: Does stopping propagation prevent navigation?

Not necessarily; default action and propagation are separate. Use preventDefault where appropriate.

Scenario: A list update loses keyboard focus.

Preserve stable elements or deliberately restore focus to a meaningful surviving control.

Debugging: The handler fires twice after navigation.

Inspect repeated registration and missing cleanup, including captured state that remains reachable.

Output-based: What does the worked example produce, and under which runtime?

A click on text or an icon inside a matching button identifies that button. The containing list must exist before registering the listener; ids remain untrusted inputs at the server boundary.

Coding: What would you implement and verify next?

Build a local notes app with a plain-text editor, explicit save state, JSON export, and safe handling of corrupted saved data. Do not present browser storage as a multi-device database.

Architecture and production: Which internal boundary needs evidence?

Events can pass through capture, target, and bubble phases. Shadow DOM can retarget events. Registering listeners repeatedly can retain state and cause duplicate work. Most DOM updates do not automatically preserve focus after replacing an element.

[Difficulty-based interview bank](#) · [Scenario practice](#) · [Debugging library](#)

Related Concepts

React identity, state, effects, and resilient interfaces, Authentication, authorization, and web security boundaries

Next: React identity, state, effects, and resilient interfaces

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Node.js runtime, resources, and asynchronous services

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: JavaScript values, scope, functions, and collections, Promises, event loops, and bounded concurrency

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Node executes JavaScript with host capabilities such as files, networking, and processes rather than a browser DOM.

Concept reference and priorities

Runtime fundamentals

P0 · Essential / Master · reference

Node executes JavaScript with host capabilities such as files, networking, and processes rather than a browser DOM.

V8

P2 · Useful · reference

V8 parses and executes JavaScript and manages memory; Node combines it with other runtime components.

Modules

P0 · Essential / Master · reference

Modules define explicit dependency boundaries; loading behavior depends on ESM or CommonJS configuration.

CommonJS

P1 · ☐ Highly Important · reference

CommonJS uses `require` and `module.exports` with its own loading and interoperability behavior.

ESM

P0 · Essential / Master · worked-example

ESM uses `import` and `export`; package type and extensions affect how Node interprets files.

npm

P0 · Essential / Master · reference

npm manages package metadata and dependency installation; a lockfile makes the chosen graph reproducible.

package.json

P0 · Essential / Master · reference

The manifest defines scripts, module type, dependency ranges, and supported runtime expectations.

File system

P1 · ☐ **Highly Important** · **worked-example**

Filesystem APIs operate on paths and data; prefer asynchronous work in request handlers and constrain user-influenced paths.

Events

P1 · ☐ **Highly Important** · **worked-example**

Runtime events announce lifecycle changes; handle required error events and remove unnecessary listeners.

EventEmitter

P1 · ☐ **Highly Important** · **worked-example**

EventEmitter invokes registered listeners according to its contract; asynchronous listener failures require deliberate handling.

Buffers

P2 · **Useful** · **worked-example**

Buffers represent binary bytes rather than Unicode text and require explicit encoding decisions.

Streams

P1 · ☐ **Highly Important** · **worked-example**

Streams process data incrementally; backpressure prevents producers from overwhelming consumers.

HTTP

P0 · **Essential / Master** · **reference**

Node exposes request and response primitives, while a framework can organize validation and routing above them.

Networking

P2 · **Useful** · **reference**

Sockets and connection pools consume finite resources; timeouts and lifecycle cleanup matter.

Process

P1 · ☐ **Highly Important** · **reference**

The process has signals, exit status, resources, and environment; shutdown must account for in-flight work.

Environment variables

P0 · **Essential / Master** · **reference**

Environment variables configure deployment; parse required values at startup and keep secrets out of logs.

Child processes

P3 · **Advanced / Specialized** · **reference**

Child processes run separate programs; avoid interpolating untrusted data into shell command strings.

Worker threads

P2 · **Useful** · **reference**

Workers can run CPU-heavy JavaScript outside the request thread, with messaging and resource costs.

Cluster

P3 · Advanced / Specialized · reference

Cluster supports multiple Node processes; process-local state is not a shared authoritative database.

Event loop

P0 · Essential / Master · reference

Node coordinates asynchronous work through runtime phases and queues; expensive synchronous work blocks progress.

Async programming

P0 · Essential / Master · worked-example

Handle ordering, failures, bounded concurrency, and cleanup explicitly.

Error handling

P0 · Essential / Master · worked-example

Separate expected domain errors from unexpected runtime failures and preserve useful diagnostic context.

Performance

P1 · ☐ Highly Important · reference

Profile latency, CPU, allocation, and resource contention with a representative workload.

Security

P0 · Essential / Master · reference

Constrain inputs, file paths, commands, secrets, dependency access, and network capabilities.

Testing

P0 · Essential / Master · reference

Exercise contracts with unit, integration, and real-network checks appropriate to the boundary.

Why Does It Exist?

One blocked request thread can delay many unrelated clients. Runtime knowledge helps decide whether work should be streamed, queued, moved to a worker, or made smaller.

How Does It Work?

Separate JavaScript execution from asynchronous I/O progress. A function declared `async` can still block while computing. Stream large data with backpressure and use pipeline so errors and cleanup compose. Limit database and remote-service concurrency instead of creating unbounded Promises. Define shutdown as stop accepting, drain to a deadline, close dependencies, and terminate remaining work.

Examples

1. Trace the contract

```
import {createReadStream, createWriteStream} from 'node:fs';
import {pipeline} from 'node:stream/promises';
await pipeline(createReadStream('input.txt'), createWriteStream('copy.txt'));
console.log('copy complete');
```

Expected behavior and runtime: With an existing input.txt and writable destination, copy.txt contains the same bytes. A missing input rejects pipeline and must be reported at the application boundary. This creates or overwrites the destination in the chosen directory.

2. Extend and stress the contract

Intermediate: stream a large file and reproduce an input error. Advanced: profile CPU blocking under concurrent requests. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Executable boundary: EventEmitter is synchronous

EventEmitter invokes registered listeners synchronously during emit. once removes its listener after the first event. An unhandled error event throws; it is not an ordinary ignored event.

Concepts: Events, EventEmitter, Error handling.

```
const {EventEmitter}=await import('node:events');
const bus=new EventEmitter(),order=[];
bus.once('saved',()=>order.push('listener'));order.push('before');
bus.emit('saved');order.push('after');bus.emit('saved');
assert.deepEqual(order,['before','listener','after']);
assert.throws(()=>new EventEmitter().emit('error',new Error('offline')),'/offline/');
bus.on('error',error=>order.push(error.message));bus.emit('error',new Error('handled'));
assert.equal(order.at(-1),'handled');
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. Listener runs before emit returns and only once. Unhandled error throws; a registered error listener receives it.

Interview: Does emit await an async listener?

Answer: No. It invokes the function; returned Promises need an explicit error and completion policy. Consider a queue or awaited operation when durable acknowledgement is required.

Change and verify: Make a listener reject. Add a documented rejection policy and verify the failure is observed instead of becoming an unhandled rejection.

Standalone executable.

Executable boundary: Stream teardown and bytes

pipeline coordinates stream completion and destroys the chain when a stage fails. Buffers contain bytes; character count is not necessarily byte count for UTF-8.

Concepts: Streams, Buffers, Async programming.

```
const {Readable,Writable}=await import('node:stream');
const {pipeline}=await import('node:stream/promises');
const
  bytes=Buffer.from('₹');assert.equal(bytes.length,3);assert.equal(bytes.toString(),'₹');
let chunks=0;const source=Readable.from(['one','two']);
const sink=new Writable({write(chunk,encoding,callback){chunks++;callback(new Error('disk full'))}});
await assert.rejects(pipeline(source,sink),'/disk full/');
assert.equal(source.destroyed,true);assert.equal(sink.destroyed,true);assert.equal(chunks,1);
```

Runtime and expected behavior: Node 24 ESM; import assert from node:assert/strict. The currency symbol occupies three UTF-8 bytes. The first failed write rejects pipeline and destroys both ends.

Interview: Why use pipeline instead of only source.pipe(destination)?

Answer: pipeline coordinates completion, error propagation and teardown across stages. Choose buffering/objectMode/highWaterMark for the actual workload; this small fixture is not a memory benchmark.

Change and verify: Use a slow sink and record write/drain behavior. Bound bytes buffered and abort a large transfer; verify all resources close.

Standalone executable.

Under the Hood

Some I/O uses operating-system mechanisms and some work uses a runtime worker pool. Worker threads are separate from that pool. A saturated pool or synchronous CPU loop can both increase latency, but need different evidence and fixes.

Real-World Usage

Add correlation-aware logs and a bounded report operation to the workspace. Compare concurrent request latency before and after moving CPU work away from the request thread.

Common Mistakes

- Reading a huge file synchronously per request.
- Leaving listeners or handles alive after a job completes.
- Using a process-local mutex as proof of multi-instance consistency.

Best Practices

Add correlation-aware logs and a bounded report operation to the workspace. Compare concurrent request latency before and after moving CPU work away from the request thread. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: inspect package type and run a matching module.
- Intermediate: stream a large file and reproduce an input error.
- Advanced: profile CPU blocking under concurrent requests.
- Challenge: terminate a service during a slow request and verify its documented drain deadline.

Mini Project

Build a report worker with a bounded queue, job status, and cancellation policy. Provide measurements and document where durable storage would be required.

Project ladder and implementation status

Interview Questions

Beginner: Does Node supply `document.querySelector`?

No. The browser DOM is not part of the ordinary Node host.

Intermediate: Why is pipeline preferable to unmanaged piping?

It coordinates stream completion, errors, and cleanup through a single outcome.

Advanced: Does `await` move CPU work to a worker?

No. Explicit worker or process execution is needed for that change.

Scenario: Latency rises during export.

Profile CPU and resource queues; use incremental I/O or a bounded worker according to the measured bottleneck.

Debugging: Shutdown hangs.

Inspect active handles, connections, timers, and unbounded in-flight work; establish a deadline.

Output-based: What does the worked example produce, and under which runtime?

With an existing input.txt and writable destination, copy.txt contains the same bytes. A missing input rejects pipeline and must be reported at the application boundary. This creates or overwrites the destination in the chosen directory.

Coding: What would you implement and verify next?

Build a report worker with a bounded queue, job status, and cancellation policy. Provide measurements and document where durable storage would be required.

Architecture and production: Which internal boundary needs evidence?

Some I/O uses operating-system mechanisms and some work uses a runtime worker pool. Worker threads are separate from that pool. A saturated pool or synchronous CPU loop can both increase latency, but need different evidence and fixes.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Express services, validation, and protected REST routes, Containers, CI/CD, and operating a web service, Behavioral testing across units, databases, and browsers

Next: Express services, validation, and protected REST routes

Preserved lessons: notes/node.md, 06-nodejs/notes.md

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Express services, validation, and protected REST routes

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: Node.js runtime, resources, and asynchronous services, REST contracts, GraphQL, and API evolution

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Create an application, mount middleware and routes, then listen through an HTTP server with a controlled lifecycle.

Concept reference and priorities

Server creation

P0 · Essential / Master · reference

Create an application, mount middleware and routes, then listen through an HTTP server with a controlled lifecycle.

Routing

P0 · Essential / Master · worked-example

Routing selects handlers by method and path; unknown API paths must not silently become frontend HTML.

Middleware

P0 · Essential / Master · reference

Middleware can inspect, transform, terminate, or delegate a request; order defines behavior.

Request/response

P0 · Essential / Master · reference

Requests contain untrusted inputs; responses implement the API status, headers, and representation contract.

Controllers

P1 · ☐ Highly Important · reference

Controllers adapt HTTP details to validated service inputs and domain outcomes.

Services

P1 · ☐ Highly Important · reference

Services own domain rules that should not depend unnecessarily on the HTTP transport.

REST APIs

P0 · Essential / Master · reference

REST uses resources and representations with coherent HTTP semantics and operational contracts.

Validation

P0 · Essential / Master · worked-example

Validate types, lengths, allowed fields, ranges, and relationships before trusted use.

Error handling

P0 · Essential / Master · reference

Centralize safe client errors while retaining server diagnostic evidence; Express 5 forwards rejected returned Promises.

Authentication

P0 · Essential / Master · reference

Resolve identity from trusted credentials rather than client-supplied role or owner fields.

Authorization

P0 · Essential / Master · worked-example

Every read and write must enforce the authenticated caller's permission for the resource.

Cookies

P1 · ☐ Highly Important · reference

Cookies are browser-managed values with scope and security attributes; they can accompany cross-site requests under applicable rules.

Sessions

P0 · Essential / Master · reference

Sessions connect credentials to server-side authority with expiration and revocation.

File uploads

P2 · Useful · reference

Limit size and count, validate content, isolate storage, and avoid trusting original filenames or declared MIME type.

Pagination

P1 · ☐ Highly Important · reference

Bound lists and define stable ordering and a cursor or offset contract.

Filtering

P1 · ☐ Highly Important · reference

Allowlist supported predicates and normalize values rather than accepting arbitrary query operators.

Sorting

P1 · ☐ Highly Important · reference

Allowlist fields and directions with a stable tie-breaker when needed.

Searching

P1 · ☐ Highly Important · reference

Specify matching behavior and bound query cost; avoid uncontrolled patterns or arbitrary regular expressions.

Rate limiting

P1 · ☐ Highly Important · reference

Choose keys and scope; a process-local limiter is not a consistent global limit across instances.

Logging

P1 · ☐ Highly Important · reference

Record useful request context and safe errors while redacting secrets and sensitive payloads.

Security

P0 · Essential / Master · reference

Layer input constraints, identity, authorization, origin defenses, safe headers, and operational limits.

API architecture

P1 · ☐ Highly Important · reference

Separate transport, domain behavior, and persistence so tests can target the relevant contract.

Production structure

P1 · ☐ Highly Important · reference

Include startup validation, repeatable builds, safe error behavior, monitoring, graceful shutdown, and deployment requirements.

Why Does It Exist?

A route that works for one happy path can expose another owner's data or fail under concurrent writes. Explicit boundaries make these failures testable.

How Does It Work?

Build fields from validated scalar inputs. Authenticate before privileged work, derive owner from the trusted identity, and include it in every persistence predicate. Middleware that responds should not continue into another responder. A catch-all frontend route must exclude API paths. Express 5 handles rejected returned Promises; callback failures still require appropriate error forwarding.

Examples

1. Trace the contract

```
app.patch('/api/tasks/:id', requireSession, async (req, res) => {
  const fields = validateTaskPatch(req.body);
  const task = await store.updateTask(req.user.id, req.params.id, fields);
  res.json(task);
});
// store uses owner and expectedVersion in one atomic update predicate.
```

Expected behavior and runtime: This integration fragment uses the workspace-style service contract; it requires app, requireSession, validator, and store implementations. The server-derived owner cannot be replaced by a body field.

2. Extend and stress the contract

Intermediate: reproduce a malformed JSON and an unsupported field. Advanced: run simultaneous same-version updates against MongoDB. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

A middleware chain is a control-flow pipeline over one request. Parsing limits apply before domain validation. Correct client recovery may depend on distinguishing invalid input, missing or unowned resource, expired session, and conflict.

Real-World Usage

Run the actual workspace ownership, malformed-body, uniqueness, session, and conflict tests. Trace which layer rejects each case.

Common Mistakes

- Spreading `request.body` into an update.
- Sending raw database or stack-trace errors to the browser.
- Checking ownership only in the interface.

Best Practices

Run the actual workspace ownership, malformed-body, uniqueness, session, and conflict tests. Trace which layer rejects each case. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: identify middleware order for one write.
- Intermediate: reproduce a malformed JSON and an unsupported field.
- Advanced: run simultaneous same-version updates against MongoDB.
- Challenge: add a filtered cursor list without permitting arbitrary client query objects.

Mini Project

Add a documented bookmark search endpoint with owner scope, bounded results, safe errors, and a contract test for another user's ids.

Project ladder and implementation status

Interview Questions

Beginner: What does middleware do?

It participates in request control flow and can terminate or delegate processing.

Intermediate: Why is server validation necessary?

Clients can send arbitrary HTTP inputs regardless of form validation.

Advanced: Does Express 5 catch every callback error?

No. Its Promise forwarding applies to returned rejected Promises; callback APIs still need proper handling.

Scenario: Two clients edit the same task.

Apply the expected version in one persisted predicate and surface a conflict for recovery.

Debugging: API returns the app shell.

Inspect fallback route order and exclude API paths from the frontend fallback.

Output-based: What does the worked example produce, and under which runtime?

This integration fragment uses the workspace-style service contract; it requires app, requireSession, validator, and store implementations. The server-derived owner cannot be replaced by a body field.

Coding: What would you implement and verify next?

Add a documented bookmark search endpoint with owner scope, bounded results, safe errors, and a contract test for another user's ids.

Architecture and production: Which internal boundary needs evidence?

A middleware chain is a control-flow pipeline over one request. Parsing limits apply before domain validation. Correct client recovery may depend on distinguishing invalid input, missing or unowned resource, expired session, and conflict.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

MongoDB modeling, querying, and persisted invariants, Authentication, authorization, and web security boundaries, Behavioral testing across units, databases, and browsers

Next: MongoDB modeling, querying, and persisted invariants

Preserved lessons: notes/express.md, 07-express-rest/express/notes.md

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

MongoDB modeling, querying, and persisted invariants

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: JavaScript values, scope, functions, and collections, REST contracts, GraphQL, and API evolution

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Documents hold structured fields and nested values; model them around access and ownership patterns.

Concept reference and priorities

Documents

P0 · Essential / Master · reference

Documents hold structured fields and nested values; model them around access and ownership patterns.

Collections

P0 · Essential / Master · reference

Collections group documents and can have validation rules and indexes.

BSON

P1 · ☐ Highly Important · reference

BSON is MongoDB's binary representation with types beyond JSON, including ObjectId and dates.

CRUD

P0 · Essential / Master · worked-example

Create, read, update, and delete must preserve domain constraints and caller scope.

Operators

P1 · ☐ Highly Important · worked-example

Operators express matching and updates; build permitted expressions rather than accepting arbitrary input operators.

Querying

P0 · Essential / Master · reference

A query selects documents according to predicates, available indexes, and execution behavior.

Projection

P1 · ☐ Highly Important · reference

Projection limits fields returned; it can reduce transfer and prevent accidental disclosure.

Sorting

P1 · ☐ Highly Important · reference

Sort determines order; include a tie-breaker when pagination requires stable identity.

Pagination

P1 · ☐ Highly Important · reference

Cursor pagination continues from a defined position; offset pagination can shift when earlier rows change.

Indexes

P0 · Essential / Master · reference

Indexes support selected access patterns and enforce uniqueness when configured; they add write and storage cost.

Aggregation

P1 · ☐ Highly Important · reference

A pipeline transforms a stream of documents through ordered stages such as match, group, and project.

Transactions

P1 · ☐ Highly Important · reference

Multi-document transactions coordinate invariants when one atomic document operation cannot express them; deployment support matters.

Schema design

P0 · Essential / Master · reference

Define required types and relationships despite flexible storage; schema evolution needs intentional migration.

Data modeling

P0 · Essential / Master · worked-example

Embedding and references trade read locality, growth, duplication, and independent lifecycle.

Replication

P2 · Useful · reference

Replica sets provide multiple copies and failover behavior; read/write concern choices affect guarantees.

Performance

P1 · ☐ Highly Important · reference

Inspect explain plans and representative data; document and query shapes matter more than a database label.

Why Does It Exist?

The database must establish invariants that survive multiple clients and processes. An application read followed by a write is vulnerable to another writer acting between them.

How Does It Work?

Begin with reads, writes, growth, owner scope, and consistency requirements. Embed bounded data updated together; reference independently growing or shared entities. Make uniqueness a real unique index. For edits, include owner and expected version in one update predicate and increment the version in that operation. A single-document write is atomic; multiple dependent writes may need a transaction or a different model.

Examples

1. Trace the contract

```
// mongosh example: first create this document
db.tasks.insertOne({_id: 'demo', owner: 'a', version: 0, status: 'todo'});
db.tasks.updateOne(
  {_id: 'demo', owner: 'a', version: 0},
  {$set: {status: 'done'}, $inc: {version: 1}}
);
// Repeating the same version predicate matches zero documents.
```

Expected behavior and runtime: In a disposable MongoDB database, the first update matches one record and advances version to 1. Repeating version 0 matches none. Do not run sample inserts in a production database.

2. Extend and stress the contract

Intermediate: compare examined and returned counts with a representative dataset. Advanced: demonstrate exactly one winner for two version-0 updates. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Indexes narrow the search space but do not make all predicates inexpensive. An explain plan should compare examined documents and keys with returned results. Aggregation order affects intermediate data size and meaning. Transactions require an appropriate deployment and do not replace explicit input or ownership checks.

Real-World Usage

Use the existing task conflict and unique-email tests with a real MongoDB process. Compare a bounded list with an expense aggregate across all account records.

Common Mistakes

- Using a read-before-insert duplicate check without a unique index.
- Embedding an unbounded growing history inside one document.
- Treating flexible schema as permission to accept arbitrary shapes.

Best Practices

Use the existing task conflict and unique-email tests with a real MongoDB process. Compare a bounded list with an expense aggregate across all account records. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: write CRUD with an owner predicate.
- Intermediate: compare examined and returned counts with a representative dataset.
- Advanced: demonstrate exactly one winner for two version-0 updates.
- Challenge: choose a model for inventory reservation and state how expiration and payment interact.

Mini Project

Extend the task board with a history model. Define the growth limit, atomic edit behavior, index needs, migration, and conflict test.

Project ladder and implementation status

Interview Questions

Beginner: Are MongoDB documents just JSON?

No. BSON supports additional types and storage semantics.

Intermediate: Does an index make every query fast?

No. Predicate, ordering, cardinality, and workload determine its usefulness.

Advanced: When is a transaction needed?

When an invariant spans dependent writes that cannot be expressed safely as one atomic document operation.

Scenario: Two users buy the last item.

Use a conditional persisted decrement or reservation predicate; a read-before-write check can race.

Debugging: Duplicate registration bypasses validation.

Establish the unique index and handle its duplicate-key error safely.

Output-based: What does the worked example produce, and under which runtime?

In a disposable MongoDB database, the first update matches one record and advances version to 1. Repeating version 0 matches none. Do not run sample inserts in a production database.

Coding: What would you implement and verify next?

Extend the task board with a history model. Define the growth limit, atomic edit behavior, index needs, migration, and conflict test.

Architecture and production: Which internal boundary needs evidence?

Indexes narrow the search space but do not make all predicates inexpensive. An explain plan should compare examined documents and keys with returned results. Aggregation order affects intermediate data size and meaning. Transactions require an appropriate deployment and do not replace explicit input or ownership checks.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Mongoose schemas, models, and database behavior, Relational modeling, SQL queries, and transactions, System design, consistency, and failure tradeoffs

Next: Mongoose schemas, models, and database behavior

Preserved lessons: [notes/mongodb.md](#), [08-databases/mongodb/notes.md](#)

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Mongoose schemas, models, and database behavior

Priority: P1 — ☐ Highly Important

Difficulty: Intermediate

Importance: 4/5

Prerequisites: MongoDB modeling, querying, and persisted invariants, Node.js runtime, resources, and asynchronous services

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Connect deliberately, validate configuration, and manage startup failure and shutdown.

Concept reference and priorities

Connection

P1 · ☐ Highly Important · **reference**

Connect deliberately, validate configuration, and manage startup failure and shutdown.

Schema

P1 · ☐ Highly Important · **worked-example**

A schema describes casting, defaults, validation, middleware, and model behavior; it is distinct from database validation.

Models

P1 · ☐ Highly Important · **worked-example**

Models provide collection operations built from a schema and connection.

Validation

P1 · ☐ Highly Important · **reference**

Validation rejects selected invalid values; update validation has specific supported operators and context rules.

Middleware

P2 · **Useful** · **reference**

Middleware surrounds selected operations; document and query middleware have different this and lifecycle behavior.

Methods

P2 · **Useful** · **reference**

Document methods attach behavior to a hydrated document; avoid arrows when document this is required.

Statics

P2 · **Useful** · **reference**

Statics expose model-level behavior such as a carefully scoped query helper.

Virtuals

P2 · Useful · reference

Virtuals compute or expose values without ordinary persisted fields; serialization options affect visibility.

Populate

P2 · Useful · reference

Populate resolves references through additional query behavior; bound cost and avoid leaking fields.

References

P1 · ☐ Highly Important · reference

References connect identities but do not automatically provide foreign-key enforcement.

Transactions

P2 · Useful · reference

Use a supported deployment and shared session for coordinated operations; avoid unsupported parallel transaction patterns.

Indexes

P1 · ☐ Highly Important · worked-example

Schema index declarations describe desired indexes; verify actual database creation and migrations.

Aggregation

P2 · Useful · reference

Aggregation operates through pipeline semantics that differ from hydrated-document methods and query casting.

Why Does It Exist?

Mongoose reduces repeated model code, but its convenience can hide the distinction between application checks and database invariants.

How Does It Work?

Choose a schema, validate permitted fields at the HTTP boundary, and use database indexes for uniqueness. `unique` is an index declaration rather than an ordinary field validator. Update validators need deliberate options and have limitations; a `findOneAndUpdate` does not behave identically to fetching a document and calling `save`. `lean` returns plain values without normal hydration features.

Examples

1. Trace the contract

```
const schema = new mongoose.Schema({
  owner: {type: mongoose.Schema.Types.ObjectId, required:true},
  title: {type:String, required:true, maxlength:120},
  version: {type:Number, required:true, default:0}
});
schema.index({owner:1, title:1}, {unique:true});
const Task = mongoose.model('Task', schema);
```

Expected behavior and runtime: This fragment requires a connected mongoose import. The unique index must actually exist in the database; duplicate writes surface database errors rather than ordinary title validation errors.

2. Extend and stress the contract

Intermediate: compare lean and hydrated reads. Advanced: test update-validation and middleware behavior for the exact method used. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Hydration creates richer document instances with change tracking. Query middleware, document middleware, populate, and lean alter what executes and what values are returned. Inspect the API-specific documentation instead of assuming every hook runs on every update.

Real-World Usage

Build an optional Mongoose persistence adapter for the same task-store contract and run the existing ownership and concurrency cases against it.

Common Mistakes

- Assuming `unique:true` is enough before an index has been created.
- Using save hooks as though they run for every query update.
- Populating unrestricted related documents for an unauthorized caller.

Best Practices

Build an optional Mongoose persistence adapter for the same task-store contract and run the existing ownership and concurrency cases against it. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: distinguish schema, model, document, and collection.
- Intermediate: compare lean and hydrated reads.
- Advanced: test update-validation and middleware behavior for the exact method used.
- Challenge: migrate an index safely and demonstrate a real concurrent uniqueness failure.

Mini Project

Implement a Mongoose adapter as a separate optional package. Document connection, validation, indexes, hook semantics, and parity with the native-driver contract.

Project ladder and implementation status

Interview Questions

Beginner: Does Mongoose replace MongoDB?

No. It provides an application modeling layer over MongoDB.

Intermediate: Is unique a Mongoose validator?

No. It declares an index requirement; duplicate-key behavior is enforced by the database.

Advanced: What can lean omit?

Hydrated document behavior such as ordinary getters, virtual processing, and document methods depending on configuration.

Scenario: Invalid values enter through an update.

Inspect update-validation options, supported operators, boundary validation, and database rules.

Debugging: A password save hook did not run.

Check whether the code used a query update rather than document save; make hashing ownership explicit.

Output-based: What does the worked example produce, and under which runtime?

This fragment requires a connected mongoose import. The unique index must actually exist in the database; duplicate writes surface database errors rather than ordinary title validation errors.

Coding: What would you implement and verify next?

Implement a Mongoose adapter as a separate optional package. Document connection, validation, indexes, hook semantics, and parity with the native-driver contract.

Architecture and production: Which internal boundary needs evidence?

Hydration creates richer document instances with change tracking. Query middleware, document middleware, populate, and lean alter what executes and what values are returned. Inspect the API-specific documentation instead of assuming every hook runs on every update.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Express services, validation, and protected REST routes, Behavioral testing across units, databases, and browsers

Next: Express services, validation, and protected REST routes

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Relational modeling, SQL queries, and transactions

Priority: P1 — ☐ Highly Important

Difficulty: Intermediate

Importance: 4/5

Prerequisites: REST contracts, GraphQL, and API evolution

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Relational data is represented through relations with explicit columns and constraints; SQL expresses operations over these sets.

Concept reference and priorities

Fundamentals

P1 · ☐ Highly Important · reference

Relational data is represented through relations with explicit columns and constraints; SQL expresses operations over these sets.

MySQL

P2 · Useful · reference

MySQL is a relational database with engine and version-specific capabilities; select the actual documentation and isolation contract.

PostgreSQL

P1 · ☐ Highly Important · reference

PostgreSQL supports relational queries, constraints, transactions, and additional types; syntax can differ from MySQL.

CRUD

P1 · ☐ Highly Important · worked-example

Insert, select, update, and delete should use parameters and enforce caller scope.

Joins

P1 · ☐ Highly Important · reference

Joins combine rows according to predicates; one-to-many relationships can multiply rows.

Subqueries

P2 · Useful · reference

Subqueries supply values or relations to another query; correlated work and plans can affect performance.

CTE

P2 · Useful · reference

A common table expression names a query result within a statement; optimization behavior is database-dependent.

Window functions

P2 · Useful · reference

Window functions calculate across related rows while retaining individual rows, such as rank or running totals.

Indexes

P1 · ☐ Highly Important · reference

Indexes accelerate selected lookups and ordering while adding write and storage cost.

Transactions

P1 · ☐ Highly Important · reference

Transactions group operations with defined isolation behavior; atomicity does not mean all concurrent anomalies disappear.

Constraints

P1 · ☐ Highly Important · worked-example

Primary keys, unique, foreign keys, not-null, and check constraints protect selected invariants in the database.

Normalization

P1 · ☐ Highly Important · reference

Normalization reduces problematic redundancy; deliberate denormalization needs update and consistency rules.

Query optimization

P2 · Useful · reference

Inspect explain plans with representative cardinality, predicates, ordering, and constraints.

Why Does It Exist?

A schema can make invalid relationships impossible to store, while joins answer questions spanning entities. SQL and document databases both require good modeling and explicit concurrency rules.

How Does It Work?

Start from entities and invariants, then define keys and constraints before adding query code. Use parameterized statements with a driver; placeholders vary across databases. A join can produce several rows per parent, so aggregating after the join must account for multiplicity. Isolation levels determine which concurrent observations are permitted; retries may be required for serialization failures.

Examples

1. Trace the contract

```
CREATE TABLE expenses (  
  id BIGINT PRIMARY KEY,  
  owner_id BIGINT NOT NULL,  
  amount_cents BIGINT NOT NULL CHECK (amount_cents > 0)  
);  
INSERT INTO expenses VALUES (1, 10, 1234), (2, 10, 100);  
SELECT owner_id, SUM(amount_cents) AS total_cents
```

```
FROM expenses GROUP BY owner_id;
```

Expected behavior and runtime: On a compatible relational engine, owner 10 has total_cents 1334. This is a disposable schema example, not a migration for the existing MongoDB workspace. Check the target engine's constraint support and integer range.

2. Extend and stress the contract

Intermediate: compare inner and left joins using a parent with no children. Advanced: compute a running total with a window function and explicit order. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Query planners choose access paths using available statistics and indexes. Foreign keys protect relationships but do not automatically authorize a query. MongoDB also supports validation and transactions; database selection should follow data and workload requirements rather than simplistic SQL-versus-NoSQL claims.

Real-World Usage

Model a job portal with applicants, jobs, and applications. Enforce unique applicant/job application pairs and scope queries to the caller.

Common Mistakes

- Interpolating a request value into SQL text.
- Summing parent amounts after a one-to-many join without accounting for duplicates.
- Assuming a transaction uses the same isolation rules on every engine.

Best Practices

Model a job portal with applicants, jobs, and applications. Enforce unique applicant/job application pairs and scope queries to the caller. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: write select and update with an owner condition.
- Intermediate: compare inner and left joins using a parent with no children.
- Advanced: compute a running total with a window function and explicit order.
- Challenge: reproduce and document a concurrency anomaly at the chosen isolation level.

Mini Project

Create a relational adapter for expense reporting with schema, parameterized queries, constraints, explain evidence, and migration rollback planning.

Project ladder and implementation status

Interview Questions

Beginner: What does a foreign key establish?

A permitted relationship to referenced rows, subject to configured rules.

Intermediate: Why can a join duplicate a total?

A parent is repeated for each matching child; aggregate at the intended grain.

Advanced: Does ACID promise no application bugs?

No. Domain invariants, isolation choices, and retry behavior still need correct design.

Scenario: A balance check passes twice concurrently.

Use a conditional update or suitable transaction/locking design, then test the real engine.

Debugging: A query is slow despite an index.

Inspect the actual plan, selectivity, ordering, statistics, and returned cardinality.

Output-based: What does the worked example produce, and under which runtime?

On a compatible relational engine, owner 10 has total_cents 1334. This is a disposable schema example, not a migration for the existing MongoDB workspace. Check the target engine's constraint support and integer range.

Coding: What would you implement and verify next?

Create a relational adapter for expense reporting with schema, parameterized queries, constraints, explain evidence, and migration rollback planning.

Architecture and production: Which internal boundary needs evidence?

Query planners choose access paths using available statistics and indexes. Foreign keys protect relationships but do not automatically authorize a query. MongoDB also supports validation and transactions; database selection should follow data and workload requirements rather than simplistic SQL-versus-NoSQL claims.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

MongoDB modeling, querying, and persisted invariants, System design, consistency, and failure tradeoffs

Next: MongoDB modeling, querying, and persisted invariants

Preserved lessons: [notes/mysql.md](#), [08-databases/sql/notes.md](#)

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Redis caching, messaging, and distributed coordination

Priority: P2 — Useful

Difficulty: Intermediate

Importance: 3/5

Prerequisites: Node.js runtime, resources, and asynchronous services, System design, consistency, and failure tradeoffs

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Redis provides data structures with operational persistence and replication choices; memory is not automatically durable authority.

Concept reference and priorities

Fundamentals

P2 · Useful · reference

Redis provides data structures with operational persistence and replication choices; memory is not automatically durable authority.

Caching

P1 · ☐ Highly Important · worked-example

A cache reuses derived values with explicit scope, freshness, expiration, and invalidation.

Sessions

P2 · Useful · reference

Shared session storage can coordinate identity across instances when expiration, revocation, and availability rules are defined.

Pub/Sub

P2 · Useful · reference

Pub/Sub delivers to current subscribers and is not a durable replayable queue for disconnected consumers.

Rate limiting

P2 · Useful · reference

Atomic shared counters or scripts can implement a chosen limit policy; define windows, keys, and failure mode.

Queues

P2 · Useful · reference

Durable job systems need acknowledgement, retry, idempotency, and recovery, beyond merely pushing a list value.

Distributed locks

P3 · Advanced / Specialized · reference

Locks have expiry and ownership constraints; stale holders may need fencing enforced by the protected resource.

Why Does It Exist?

Caching can remove repeated expensive work, but a misplaced key can expose data or return stale decisions. Shared coordination must handle failures and multiple processes.

How Does It Work?

For cache-aside, read cache, fetch the authoritative value on a miss, and store a bounded-lifetime representation. Include account scope and all relevant inputs. Define invalidation after writes and behavior when Redis is unavailable. Use a token for lock ownership, compare it atomically before release, and consider stale-holder writes after expiry. Do not claim a lock makes an external payment exactly once.

Examples

1. Trace the contract

```
# Redis CLI, disposable local instance
SET "public:catalog:v1" '{"items":[]}' EX 60
GET "public:catalog:v1"
TTL "public:catalog:v1"
```

Expected behavior and runtime: GET returns the stored JSON text until expiration or deletion. TTL is remaining seconds; expiration is not a domain authorization check. This example deliberately caches public data.

2. Extend and stress the contract

Intermediate: separate public and account-specific key design. Advanced: reproduce a stale-cache race after a write. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Expiration, eviction, persistence, and replication affect what survives load or failure. Pub/Sub lacks replay for missed messages. Distributed-lock correctness depends on assumptions, timing, and how the protected system handles stale operations.

Real-World Usage

Cache a public aggregate with a measured latency goal and a defined stale-data budget. Keep account authorization and critical writes against the authoritative store.

Common Mistakes

- Using one cache key for every logged-in account.
- Relying on Pub/Sub to replay missed business events.
- Deleting a lock that another holder acquired after expiry.

Best Practices

Cache a public aggregate with a measured latency goal and a defined stale-data budget. Keep account authorization and critical writes against the authoritative store. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: observe a short TTL expire.
- Intermediate: separate public and account-specific key design.
- Advanced: reproduce a stale-cache race after a write.
- Challenge: document cache-unavailable behavior and a stale lock holder scenario.

Mini Project

Build an optional Redis cache adapter for a public read model. Acceptance: bounded keys and TTL, invalidation, account separation, failure behavior, and measured benefit.

Project ladder and implementation status

Interview Questions

Beginner: Is Redis always just a cache?

No. It supports several data structures and operational modes, whose guarantees must be selected deliberately.

Intermediate: What belongs in a cache key?

Every input and scope that changes the represented value.

Advanced: Why might fencing be required?

A holder can keep working after its lease expires; the resource must reject stale authority.

Scenario: A subscriber misses a message while disconnected.

Use durable replay or a queue/stream when missed work must be recovered.

Debugging: A fresh write returns stale data.

Trace invalidation ordering, read races, key inputs, and the chosen freshness contract.

Output-based: What does the worked example produce, and under which runtime?

GET returns the stored JSON text until expiration or deletion. TTL is remaining seconds; expiration is not a domain authorization check. This example deliberately caches public data.

Coding: What would you implement and verify next?

Build an optional Redis cache adapter for a public read model. Acceptance: bounded keys and TTL, invalidation, account separation, failure behavior, and measured benefit.

Architecture and production: Which internal boundary needs evidence?

Expiration, eviction, persistence, and replication affect what survives load or failure. Pub/Sub lacks replay for missed messages. Distributed-lock correctness depends on assumptions, timing, and how the protected system handles stale operations.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Authentication, authorization, and web security boundaries, WebSockets, SSE, and recoverable real-time delivery

Next: Authentication, authorization, and web security boundaries

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Cloud deployment, scaling, and recovery choices

Priority: P2 — Useful

Difficulty: Intermediate

Importance: 3/5

Prerequisites: Containers, CI/CD, and operating a web service

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Cloud platforms offer managed resources under explicit availability, pricing, and responsibility models.

Concept reference and priorities

Cloud fundamentals

P2 · Useful · reference

Cloud platforms offer managed resources under explicit availability, pricing, and responsibility models.

Deployment architecture

P1 · ☐ Highly Important · worked-example

Map clients, compute, stores, networks, secrets, and failure boundaries before selecting a service.

Compute

P2 · Useful · reference

Compute runs workloads under resource limits and lifecycle rules, whether instances, containers, or functions.

Storage

P1 · ☐ Highly Important · reference

Object, block, and filesystem storage have different access and durability contracts.

Databases

P1 · ☐ Highly Important · reference

Managed databases reduce some operating work, while schema, queries, permissions, and recovery remain application responsibilities.

CDN

P2 · Useful · reference

A CDN distributes cacheable content closer to clients; private data and invalidation need intentional policies.

DNS

P1 · ☐ Highly Important · reference

DNS directs names to services with propagation and caching behavior relevant to migrations.

Environment management

P1 · [□ Highly Important](#) · [reference](#)

Separate development, test, and production identities, configuration, and data.

Secrets

P0 · [Essential](#) / [Master](#) · [reference](#)

Provide credentials through a controlled secret mechanism, with least privilege and rotation.

Scaling

P2 · [Useful](#) · [reference](#)

Scale the measured bottleneck while accounting for shared-state, connection, and dependency limits.

Why Does It Exist?

Moving a service to a cloud provider changes operational boundaries, not application correctness. A managed database does not choose good indexes or authorize users for you.

How Does It Work?

Draw the request path and annotate ownership, data sensitivity, latency, and failure behavior. Keep internal stores inaccessible from the public Internet where practical. Separate public static content from personalized API data. State what happens if a zone, dependency, or deployment fails and how restored data will be verified. Estimate resource usage and budget with the provider's current pricing rather than fixed historical numbers.

Examples

1. Trace the contract

Browser -> CDN/public assets
Browser -> HTTPS entry point -> application compute -> private database
Application compute -> controlled secret access
Logs/metrics -> observation system
Backups -> separately controlled recovery storage

Expected behavior and runtime: This is a provider-neutral architecture sketch, not a provisioned environment. Each arrow needs network, identity, lifetime, and failure rules before deployment.

2. Extend and stress the contract

Intermediate: separate environment identities and secrets. Advanced: define recovery point and recovery time goals from product requirements. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Scaling compute can multiply database connections and shift the bottleneck. Managed availability is bounded by configuration and service contracts. Backups, replicas, and multi-region copies solve different failure and recovery problems.

Real-World Usage

Write a deployment decision record for the workspace: one small service, managed MongoDB or a self-managed store, public assets, TLS, secret access, backup and recovery requirements.

Common Mistakes

- Exposing a database to make setup convenient.
- Assuming replicas replace backup against accidental data deletion.
- Scaling instances without budgeting connections and background work.

Best Practices

Write a deployment decision record for the workspace: one small service, managed MongoDB or a self-managed store, public assets, TLS, secret access, backup and recovery requirements. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: label public and private components.
- Intermediate: separate environment identities and secrets.
- Advanced: define recovery point and recovery time goals from product requirements.
- Challenge: simulate dependency loss and demonstrate an honest user-facing recovery state.

Mini Project

Produce a provider-neutral deployment packet and then an optional provider-specific implementation with measured cost assumptions, health, rollback, and restore evidence.

Project ladder and implementation status

Interview Questions

Beginner: What does managed mean?

Some operations are supplied by the provider; the exact responsibility boundary still needs review.

Intermediate: Can a CDN cache all API responses?

No. Scope, privacy, freshness, and invalidation determine which representations are reusable.

Advanced: Why does scaling increase database pressure?

Each worker may add connections, concurrent queries, and background jobs.

Scenario: A region is unavailable.

Follow the chosen availability and recovery contract; avoid claiming instant failover without tested architecture.

Debugging: A restored service has old data.

Inspect recovery point, backup age, restore verification, and the actual write path.

Output-based: What does the worked example produce, and under which runtime?

This is a provider-neutral architecture sketch, not a provisioned environment. Each arrow needs network, identity, lifetime, and failure rules before deployment.

Coding: What would you implement and verify next?

Produce a provider-neutral deployment packet and then an optional provider-specific implementation with measured cost assumptions, health, rollback, and restore evidence.

Architecture and production: Which internal boundary needs evidence?

Scaling compute can multiply database connections and shift the bottleneck. Managed availability is bounded by configuration and service contracts. Backups, replicas, and multi-region copies solve different failure and recovery problems.

[Difficulty-based interview bank](#) · [Scenario practice](#) · [Debugging library](#)

Related Concepts

System design, consistency, and failure tradeoffs, Authentication, authorization, and web security boundaries

Next: [System design, consistency, and failure tradeoffs](#)

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the [verified media register](#). A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

[Local project ladder](#) · [Timed exercises](#)

Further Reading

[Curated external register](#) — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Maintainable code, architecture, and observability

Priority: P1 — ☐ Highly Important

Difficulty: Intermediate

Importance: 4/5

Prerequisites: JavaScript values, scope, functions, and collections, Git history, collaboration, and code review

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Names, small coherent contracts, explicit boundaries, and readable control flow make change easier to assess.

Concept reference and priorities

Clean code

P1 · ☐ Highly Important · reference

Names, small coherent contracts, explicit boundaries, and readable control flow make change easier to assess.

SOLID

P2 · Useful · reference

SOLID principles describe responsibility and substitutability concerns; applying them mechanically can overcomplicate small systems.

DRY

P1 · ☐ Highly Important · reference

Remove duplicated knowledge whose inconsistent evolution would cause bugs; similar-looking code can have different reasons to change.

KISS

P0 · Essential / Master · reference

Prefer a simple sufficient model that makes the important behavior visible.

YAGNI

P1 · ☐ Highly Important · reference

Delay speculative capabilities until a real requirement justifies their cost.

Design patterns

P2 · Useful · reference

Patterns name recurring solutions and tradeoffs; choose them for a concrete problem rather than a diagram collection.

Architecture patterns

P2 · Useful · reference

Architectural styles organize dependency and deployment boundaries under specific constraints.

Layered architecture

P1 · ☐ Highly Important · worked-example

Layers separate transport, domain, and persistence responsibilities when this clarifies ownership.

MVC

P2 · Useful · reference

MVC separates model, view, and controller roles, though framework interpretations vary.

Clean architecture

P2 · Useful · reference

Clean architecture directs dependencies toward domain behavior and uses adapters around external systems.

Dependency injection

P1 · ☐ Highly Important · worked-example

Pass dependencies explicitly so authority, failure behavior, and test boundaries are visible.

Error handling

P0 · Essential / Master · reference

Translate errors at meaningful boundaries without losing diagnostic context or exposing secrets.

Logging

P1 · ☐ Highly Important · reference

Logs record contextual events; structured fields help query them and redaction prevents sensitive disclosure.

Observability

P1 · ☐ Highly Important · reference

Logs, metrics, and traces provide evidence about internal behavior; useful signals are tied to concrete questions.

Documentation

P0 · Essential / Master · reference

Document contracts, decisions, setup, limitations, and verification so another developer can act without guessing.

Why Does It Exist?

A project survives when another engineer can change one rule without accidentally changing unrelated behavior. Architecture should help a concrete change rather than create layers for their own sake.

How Does It Work?

Put domain rules where they can be named and tested. Let HTTP controllers translate protocol details, services decide domain outcomes, and store adapters establish persistence behavior. Inject a store instead of reading a global singleton inside every function. Record a decision with alternatives, constraints, expected benefit, and a condition that would make you revisit it. Add telemetry to answer specific failure questions.

Examples

1. Trace the contract

```
function createTaskService(store) {
  return {
    async rename(owner, id, title, expectedVersion) {
      if (typeof title !== 'string' || !title.trim()) throw new Error('Invalid title');
      return store.updateTask(owner, id, {title:title.trim(), expectedVersion});
    }
  };
}
```

Expected behavior and runtime: The service requires a store matching the stated contract. It rejects a blank title before calling the store; persisted ownership and version safety still belong in the adapter. A production error type should preserve a safe code.

2. Extend and stress the contract

Intermediate: inject a store and simulate one expected failure. Advanced: write a decision record comparing two persistence boundaries. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Dependency direction affects which details propagate through a change. Observability correlates events through a request or job id, while metrics aggregate signals such as latency distribution and error rates. A high-cardinality label can overwhelm a metrics system even when each request is cheap.

Real-World Usage

Explain why the existing workspace separates app, validation, and store. Add a request id and safe duration/status fields without logging credentials or full payloads.

Common Mistakes

- Creating generic repositories before understanding the actual invariant.
- Hiding errors in an empty catch or returning a success-looking fallback.
- Logging every user id as an unbounded metrics label.

Best Practices

Explain why the existing workspace separates app, validation, and store. Add a request id and safe duration/status fields without logging credentials or full payloads. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: name one function by its actual contract.
- Intermediate: inject a store and simulate one expected failure.
- Advanced: write a decision record comparing two persistence boundaries.
- Challenge: diagnose a slow workflow with a correlated trace, logs, and a bounded metric.

Mini Project

Refactor one workspace domain only when it improves a demonstrated change. Include before/after behavior, a dependency diagram, and a test showing the invariant remains intact.

Project ladder and implementation status

Interview Questions

Beginner: Is shorter code always cleaner?

No. Visible intent, correct boundaries, and understandable behavior matter more than line count.

Intermediate: When should duplication remain?

When the similar code expresses independently changing knowledge and extraction would couple it unnecessarily.

Advanced: What does dependency injection establish?

Explicit dependency ownership and replaceable boundaries; it does not prove the substitute behaves like a real store.

Scenario: Every small change touches many layers.

Inspect unnecessary abstractions and ownership; simplify the boundary around the actual rule.

Debugging: A request fails with no useful context.

Propagate a correlation id, classify the error safely, and retain cause information in controlled server diagnostics.

Output-based: What does the worked example produce, and under which runtime?

The service requires a store matching the stated contract. It rejects a blank title before calling the store; persisted ownership and version safety still belong in the adapter. A production error type should preserve a safe code.

Coding: What would you implement and verify next?

Refactor one workspace domain only when it improves a demonstrated change. Include before/after behavior, a dependency diagram, and a test showing the invariant remains intact.

Architecture and production: Which internal boundary needs evidence?

Dependency direction affects which details propagate through a change. Observability correlates events through a request or job id, while metrics aggregate signals such as latency distribution and error rates. A high-cardinality label can overwhelm a metrics system even when each request is cheap.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Behavioral testing across units, databases, and browsers, REST contracts, GraphQL, and API evolution, System design, consistency, and failure tradeoffs

Next: Behavioral testing across units, databases, and browsers

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

System design, consistency, and failure tradeoffs

Priority: P1 — ☐ Highly Important

Difficulty: Intermediate

Importance: 4/5

Prerequisites: REST contracts, GraphQL, and API evolution, MongoDB modeling, querying, and persisted invariants

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Scalability is the ability to support a changing workload under stated resource and performance constraints.

Concept reference and priorities

Scalability

P1 · ☐ Highly Important · reference

Scalability is the ability to support a changing workload under stated resource and performance constraints.

Availability

P1 · ☐ Highly Important · reference

Availability describes whether useful service is accessible under defined failure conditions.

Reliability

P1 · ☐ Highly Important · reference

Reliability concerns correct operation over time, including detection, recovery, and preserved invariants.

Load balancing

P2 · Useful · reference

Load balancers distribute requests among eligible targets; health and connection behavior affect outcomes.

Caching

P1 · ☐ Highly Important · reference

Caching reuses representations under scope and freshness rules; invalidation and stampede behavior matter.

Database scaling

P2 · Useful · reference

Improve schema, queries, indexes, and workload before choosing replication or partitioning complexity.

Replication

P2 · Useful · reference

Replicas copy data with lag and failure behavior; reads need consistency requirements.

Sharding

P3 · Advanced / Specialized · reference

Sharding partitions data and introduces routing, balancing, hotspot, and cross-partition concerns.

Queues

P1 · ☐ Highly Important · reference

Queues decouple work with acknowledgment, retry, backpressure, and idempotency contracts.

Pub/Sub

P2 · Useful · reference

Publish/subscribe routes events to interested consumers; durability depends on the chosen system and configuration.

CDN

P2 · Useful · reference

CDNs distribute reusable content and require cache-key, invalidation, and privacy policies.

Rate limiting

P1 · ☐ Highly Important · reference

A limiter constrains work under explicit identity, window, deployment scope, and failure behavior.

CAP theorem

P3 · Advanced / Specialized · reference

During a network partition, a distributed system cannot guarantee both linearizable consistency and availability in the theorem's sense.

Consistency

P1 · ☐ Highly Important · worked-example

Consistency models describe which observations are permitted; choose requirements for each operation.

Distributed systems

P3 · Advanced / Specialized · reference

Multiple nodes communicate under delay, failure, and partial knowledge; local success is not a global guarantee.

High-level design

P1 · ☐ Highly Important · reference

High-level design identifies components, data flow, scale assumptions, and failure boundaries.

Low-level design

P1 · ☐ Highly Important · worked-example

Low-level design describes data models, interfaces, states, algorithms, and invariants inside those components.

Why Does It Exist?

A diagram with many boxes does not explain what the product guarantees. System design begins with users, workload, latency, consistency, and acceptable failure.

How Does It Work?

State requirements and assumptions before picking technology. Estimate workload and identify the first bottleneck. Choose an invariant such as stock never becoming negative, then trace competing writes, retries, and partial failure. For business events, writing state and then publishing can lose the event during a crash; an outbox stores the state change and event record together and a publisher retries. Consumers need deduplication because retries can repeat delivery.

Examples

1. Trace the contract

```
// Illustrative inventory contract, not a complete reservation workflow
update inventory
where productId = requestedId and available >= requestedQuantity
set available = available - requestedQuantity
// Success depends on one atomic database predicate, not a prior UI check.
```

Expected behavior and runtime: A conditional persisted update can protect a nonnegative stock invariant. A full reservation flow also needs quantity validation, expiry, payment coordination, and idempotency; this pseudocode does not implement them.

2. Extend and stress the contract

Intermediate: state consistency requirements for profile reads and purchases. Advanced: trace a crash between state commit and event publish. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Network timeout means the caller lacks certainty, not that the server did nothing. Exactly-once transport should not be assumed; a durable idempotent business transition can prevent repeated effects. CAP is not a rule to casually choose any two letters outside the relevant partition scenario.

Real-World Usage

Design a notification center, URL shortener, or inventory service. Tie each added component to a specific requirement and explain the smallest initial deployment.

Common Mistakes

- Starting with sharding before defining access patterns.
- Claiming an in-memory Set prevents duplicate effects after restart.
- Treating a timeout as proof that a write failed.

Best Practices

Design a notification center, URL shortener, or inventory service. Tie each added component to a specific requirement and explain the smallest initial deployment. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: distinguish latency and throughput with units.
- Intermediate: state consistency requirements for profile reads and purchases.
- Advanced: trace a crash between state commit and event publish.

- Challenge: test duplicate delivery and stale workers while preserving a persisted invariant.

Mini Project

Write a design packet for inventory reservation or notifications: assumptions, data model, failure matrix, recovery, sizing, and one runnable invariant test.

Project ladder and implementation status

Interview Questions

Beginner: What is a useful scale estimate?

A stated assumption with units tied to the operation or storage workload.

Intermediate: Why do queue consumers need idempotency?

Retries or replay can deliver the same work more than once.

Advanced: What does an outbox solve?

It closes the state-change/event-record gap when both are stored in the same transaction boundary; publishing still needs retries and duplicate handling.

Scenario: A payment callback repeats after restart.

Use a persisted provider operation id and an atomic state transition to prevent a second effect.

Debugging: The feed skips items during inserts.

Inspect ordering and continuation predicates; use a stable tie-breaker and define snapshot expectations.

Output-based: What does the worked example produce, and under which runtime?

A conditional persisted update can protect a nonnegative stock invariant. A full reservation flow also needs quantity validation, expiry, payment coordination, and idempotency; this pseudocode does not implement them.

Coding: What would you implement and verify next?

Write a design packet for inventory reservation or notifications: assumptions, data model, failure matrix, recovery, sizing, and one runnable invariant test.

Architecture and production: Which internal boundary needs evidence?

Network timeout means the caller lacks certainty, not that the server did nothing. Exactly-once transport should not be assumed; a durable idempotent business transition can prevent repeated effects. CAP is not a rule to casually choose any two letters outside the relevant partition scenario.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Redis caching, messaging, and distributed coordination, WebSockets, SSE, and recoverable real-time delivery, Cloud deployment, scaling, and recovery choices

Next: Redis caching, messaging, and distributed coordination

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

REST contracts, GraphQL, and API evolution

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: Internet, HTTP, and the browser, JavaScript values, scope, functions, and collections

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

REST organizes interaction around resources and representations; use coherent HTTP semantics rather than arbitrary action names.

Concept reference and priorities

REST

P0 · Essential / Master · reference

REST organizes interaction around resources and representations; use coherent HTTP semantics rather than arbitrary action names.

GraphQL

P2 · Useful · reference

GraphQL uses a typed schema and selection sets; resolvers still need authorization, cost bounds, and error contracts.

HTTP methods

P0 · Essential / Master · reference

Methods have semantics such as retrieval, replacement, and partial update; choose them consistently.

Status codes

P0 · Essential / Master · worked-example

Status codes describe response outcomes; preserve distinctions clients need for recovery.

Headers

P0 · Essential / Master · reference

Headers carry representation, caching, authentication, and other protocol metadata.

Authentication

P0 · Essential / Master · reference

Authentication establishes the caller's trusted identity from accepted credentials.

Authorization

P0 · Essential / Master · reference

Authorization establishes whether that caller may perform this operation on this resource.

Pagination

P1 · ☐ Highly Important · reference

Lists need bounded sizes and a stable continuation or offset contract.

Filtering

P1 · ☐ Highly Important · reference

Supported filter fields and operators should be explicit and validated.

Sorting

P1 · ☐ Highly Important · reference

Document ordering, allowed fields, and tie-breakers rather than accepting arbitrary expressions.

Versioning

P2 · Useful · reference

Versioning manages incompatible contract changes; keep compatibility policy visible to consumers.

Validation

P0 · Essential / Master · reference

Validate runtime shape, types, ranges, allowed fields, and domain relationships.

Error handling

P0 · Essential / Master · worked-example

Return safe structured errors with a stable code and useful message; preserve server diagnostic context separately.

Rate limiting

P1 · ☐ Highly Important · reference

Define caller identity, window policy, distributed scope, and recovery guidance.

API documentation

P0 · Essential / Master · reference

Document authentication, inputs, outputs, errors, limits, and examples for actual implemented behavior.

OpenAPI / Swagger

P1 · ☐ Highly Important · reference

OpenAPI describes HTTP contracts; Swagger is a family of tools around API descriptions rather than another protocol.

Why Does It Exist?

The client needs to know how to recover from invalid input, missing authority, and concurrent edits. An explicit contract allows independent implementation and useful integration tests.

How Does It Work?

Begin with resource identity and permitted operations. State required fields, server-owned fields, status behavior, and error representation. A 409 conflict can tell the UI to refresh before reapplying intent. Retries need idempotency: repeating a create request can duplicate work unless the server establishes a unique operation identity. GraphQL changes request composition, not trust boundaries.

Examples

1. Trace the contract

```
const error = {  
  error:{code:'VERSION_CONFLICT', message:'Refresh before editing again'}}  
};  
// Example HTTP outcome: 409 with application/json.  
// Client preserves the draft, refreshes current data, and asks for reconciliation.
```

Expected behavior and runtime: This illustrates the workspace conflict response, not an executable server. The durable version predicate decides the conflict; the response defines client recovery.

2. Extend and stress the contract

Intermediate: document invalid input, expired session, and conflict recovery. Advanced: design a cursor that includes a stable tie-breaker. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Proxies, caches, browsers, clients, and servers all interpret protocol information. An OpenAPI document is a description and can drift from code; contract tests should check important examples. GraphQL query flexibility requires limits on depth, amount of work, and authorization inside resolvers.

Real-World Usage

Write an OpenAPI contract for the implemented workspace routes and compare it with API tests. Keep list limits and full-account summary semantics explicit.

Common Mistakes

- Returning 200 for every failure.
- Publishing an API description for features the source does not implement.
- Treating GraphQL schema types as a replacement for identity and permissions.

Best Practices

Write an OpenAPI contract for the implemented workspace routes and compare it with API tests. Keep list limits and full-account summary semantics explicit. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: describe a read and create request with status and content type.
- Intermediate: document invalid input, expired session, and conflict recovery.
- Advanced: design a cursor that includes a stable tie-breaker.
- Challenge: model retried create operations and prove duplicate business effects do not occur.

Mini Project

Add an executable API contract packet for the workspace with examples, errors, and regression evidence. Extend it only when the corresponding behavior exists.

Interview Questions

Beginner: How do authentication and authorization differ?

Authentication establishes identity; authorization decides permitted actions.

Intermediate: Why bound a list?

To control response size and resource usage while defining a continuation contract.

Advanced: Does GraphQL make a query cheap?

No. Resolver work and result cardinality need explicit limits and measurement.

Scenario: A timeout is followed by a duplicate create.

The first operation may have succeeded; use a persisted idempotency contract before automatic replay.

Debugging: Documentation says a field is required but the server accepts none.

Reconcile schema and implementation and add a contract regression test.

Output-based: What does the worked example produce, and under which runtime?

This illustrates the workspace conflict response, not an executable server. The durable version predicate decides the conflict; the response defines client recovery.

Coding: What would you implement and verify next?

Add an executable API contract packet for the workspace with examples, errors, and regression evidence. Extend it only when the corresponding behavior exists.

Architecture and production: Which internal boundary needs evidence?

Proxies, caches, browsers, clients, and servers all interpret protocol information. An OpenAPI document is a description and can drift from code; contract tests should check important examples. GraphQL query flexibility requires limits on depth, amount of work, and authorization inside resolvers.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Express services, validation, and protected REST routes, Authentication, authorization, and web security boundaries, Behavioral testing across units, databases, and browsers

Next: Express services, validation, and protected REST routes

Preserved lessons: [notes/rest-api.md](#), [07-express-rest/rest-api/notes.md](#)

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Authentication, authorization, and web security boundaries

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: REST contracts, GraphQL, and API evolution, Node.js runtime, resources, and asynchronous services

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Store a password-specific salted verifier with suitable cost, not reversible plaintext encryption.

Concept reference and priorities

Password hashing

P0 · Essential / Master · reference

Store a password-specific salted verifier with suitable cost, not reversible plaintext encryption.

bcrypt

P1 · ☐ Highly Important · reference

bcrypt is a password-hashing option with cost and input-length considerations; follow current guidance for new systems.

JWT

P1 · ☐ Highly Important · reference

A signed JWT provides integrity for claims, not confidentiality; validate issuer, audience, algorithm, and expiry.

Access tokens

P1 · ☐ Highly Important · reference

Access credentials authorize a bounded scope and lifetime; keep exposure and revocation needs explicit.

Refresh tokens

P2 · Useful · reference

Refresh credentials renew access and need protected storage, rotation, reuse detection, and revocation policy.

Cookies

P0 · Essential / Master · reference

Cookie attributes control scope and handling; HttpOnly blocks ordinary script reads but does not stop all authenticated abuse.

Sessions

P0 · Essential / Master · reference

Server-side sessions support expiry and revocation; store only necessary authority and enforce expiry during lookup.

OAuth

P2 · Useful · reference

OAuth delegates access under a protocol; identity login usually also needs an appropriate identity layer such as OpenID Connect.

RBAC

P1 · ☐ Highly Important · reference

Role-based permissions group allowed actions; object ownership and tenant scope still need enforcement.

CORS

P0 · Essential / Master · reference

CORS controls browser cross-origin response access; it is not API authentication and does not stop non-browser callers.

CSRF

P0 · Essential / Master · reference

Cookie-authenticated writes need cross-site request defenses such as suitable tokens, origin checks, and SameSite policy.

XSS

P0 · Essential / Master · reference

Untrusted content executed in the page can act with the page's privileges; render data safely and constrain script execution.

SQL injection

P0 · Essential / Master · reference

Parameterized queries separate values from SQL structure; dynamic identifiers require allowlists.

NoSQL injection

P0 · Essential / Master · reference

Construct permitted query predicates from validated scalars instead of accepting arbitrary operator objects.

Rate limiting

P1 · ☐ Highly Important · reference

Limit abuse according to a defined identity and deployment scope; consider cost and recovery as well as request count.

Helmet

P1 · ☐ Highly Important · reference

Helmet sets security-related HTTP headers, but does not replace input validation or authorization.

Secure headers

P1 · ☐ Highly Important · reference

Headers can constrain browser capabilities and content interpretation; configure them for the actual deployment.

Secrets

P0 · Essential / Master · worked-example

Keep secrets out of source, logs, browser bundles, and artifacts; rotate compromised credentials.

OWASP

P1 · ☐ Highly Important · reference

OWASP offers threat-focused guidance; use it to evaluate concrete application boundaries.

Security best practices

P0 · Essential / Master · reference

Define assets, threats, trusted identities, permitted operations, and recovery; verify them with adversarial cases.

Why Does It Exist?

A valid session does not authorize access to every object. Security emerges from explicit boundaries and server-enforced invariants rather than from one token format.

How Does It Work?

Trace identity through every read and write. Never trust an owner or role supplied by the browser. Use a random session token, keep a digest server-side, expire it during lookup, and revoke it on logout. For cookie-authenticated writes, define CSRF and origin defenses for the deployment; HttpOnly and CORS alone are insufficient. Use HTTPS and appropriate cookie scope in deployment.

Examples

1. Trace the contract

```
const predicate = {_id: itemId, owner: authenticatedUserId};
// Both identity and object scope belong in the persistence predicate.
const response = {id: item.id, title: item.title};
// Deliberately omit passwordHash, session tokens, and internal authorization data.
```

Expected behavior and runtime: This boundary illustration returns only public fields from an authorized item. It requires validated identifiers and a trusted authenticatedUserId; substituting a body owner would invalidate the design.

2. Extend and stress the contract

Intermediate: attempt every endpoint as another owner. Advanced: verify session expiry independently of TTL cleanup. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Signed token payloads are usually readable encodings. Cookies can be automatically attached to requests, which motivates CSRF defenses. A compromised page can initiate requests even when it cannot read an HttpOnly cookie. Defense layers reduce different classes of risk and should not be described as interchangeable.

Real-World Usage

Run the workspace two-user tests and revocation test. Document its single-origin deployment assumption and missing production capabilities rather than calling it a finished identity service.

Common Mistakes

- Putting passwords or confidential data inside a signed JWT.
- Checking ownership only after returning data.
- Disabling TLS or broadening origin checks to make a deployment appear to work.

Best Practices

Run the workspace two-user tests and revocation test. Document its single-origin deployment assumption and missing production capabilities rather than calling it a finished identity service. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: distinguish authentication, authorization, and validation.
- Intermediate: attempt every endpoint as another owner.
- Advanced: verify session expiry independently of TTL cleanup.
- Challenge: design refresh-token reuse detection and a recovery path without inventing a custom cryptographic protocol.

Mini Project

Create a threat model for the task board covering XSS, CSRF, object access, brute-force cost, secret leakage, and operational recovery. Link each implemented control to a test or deployment requirement.

Project ladder and implementation status

Interview Questions

Beginner: Is a signed JWT encrypted?

No. Signing protects integrity; encryption is a separate capability.

Intermediate: Does CORS protect an API from curl?

No. It governs selected browser behavior, not authorization for arbitrary clients.

Advanced: Why check expiry when a TTL index exists?

TTL cleanup is asynchronous; a remaining document can already be expired.

Scenario: User A sends user B's id.

Scope the persisted operation to the trusted owner and test unchanged data for B.

Debugging: Secure cookies disappear locally.

Inspect HTTPS, cookie scope, SameSite, and deployment topology; keep production protections intact.

Output-based: What does the worked example produce, and under which runtime?

This boundary illustration returns only public fields from an authorized item. It requires validated identifiers and a trusted authenticatedUserId; substituting a body owner would invalidate the design.

Coding: What would you implement and verify next?

Create a threat model for the task board covering XSS, CSRF, object access, brute-force cost, secret leakage, and operational recovery. Link each implemented control to a test or deployment requirement.

Architecture and production: Which internal boundary needs evidence?

Signed token payloads are usually readable encodings. Cookies can be automatically attached to requests, which motivates CSRF defenses. A compromised page can initiate requests even when it cannot read an HttpOnly cookie. Defense layers reduce different classes of risk and should not be described as interchangeable.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Express services, validation, and protected REST routes, Containers, CI/CD, and operating a web service, WebSockets, SSE, and recoverable real-time delivery

Next: Express services, validation, and protected REST routes

Preserved lessons: [notes/authentication.md](#), [10-security/notes.md](#)

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

WebSockets, SSE, and recoverable real-time delivery

Priority: P2 — Useful

Difficulty: Intermediate

Importance: 3/5

Prerequisites: Promises, event loops, and bounded concurrency, REST contracts, GraphQL, and API evolution, Authentication, authorization, and web security boundaries

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

WebSockets provide bidirectional messages over a long-lived connection; application protocols still define identity and delivery.

Concept reference and priorities

WebSockets

P2 · Useful · reference

WebSockets provide bidirectional messages over a long-lived connection; application protocols still define identity and delivery.

Socket.IO

P2 · Useful · reference

Socket.IO adds its own protocol, reconnection, rooms, and acknowledgements; it is not wire-compatible with plain WebSocket clients.

Server-Sent Events

P2 · Useful · worked-example

SSE streams server-to-client events over HTTP; EventSource handles connection behavior while replay needs application design.

Notifications

P2 · Useful · reference

Notifications need ownership, stable event ids, read state, and a catch-up mechanism after disconnect.

Chat architecture

P3 · Advanced / Specialized · reference

Chat combines durable messages, authorized rooms, delivery acknowledgements, ordering, and reconnect recovery.

Presence

P3 · Advanced / Specialized · reference

Presence is typically an expiring observation of connectivity, not a durable statement that someone is actively reading.

Real-time dashboards

P2 · Useful · reference

Dashboards need bounded update frequency, backpressure, and a consistent initial snapshot or replay position.

Why Does It Exist?

A live connection can disconnect at the exact moment an event is sent. Reliable product behavior needs durable state and recovery in addition to transport.

How Does It Work?

Choose SSE for predominantly one-way updates and a bidirectional protocol when both directions need live messages. Authenticate the connection and authorize every room subscription. Persist business state before announcing it. Carry stable event ids and let a reconnecting client retrieve missed changes or refresh authoritative state. An acknowledgement is not proof that every downstream business effect is complete.

Examples

1. Trace the contract

```
const events = new EventSource('/api/events');
events.addEventListener('task-updated', event => {
  const update = JSON.parse(event.data);
  console.log(update.id, update.version);
});
// On view cleanup: events.close();
```

Expected behavior and runtime: In a browser with a matching authorized SSE endpoint, task-updated messages log the supplied id and version. This is a client fragment; the current workspace does not yet expose that endpoint.

2. Extend and stress the contract

Intermediate: close a subscription when the view unmounts. Advanced: disconnect between persistence and notification and recover the current state. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

A transport connection and a durable event log are separate things. Network partitions, process restart, and multiple publishers can change observed ordering. A snapshot followed by subscription can lose intervening writes unless a replay position or reconciliation protocol closes the gap.

Real-World Usage

Extend the task board with notifications while retaining database versions. On reconnection, refresh current tasks before applying further changes.

Common Mistakes

- Broadcasting every account's updates to every connection.
- Assuming automatic reconnect replays missed messages.
- Letting the message stream grow an unbounded UI list.

Best Practices

Extend the task board with notifications while retaining database versions. On reconnection, refresh current tasks before applying further changes. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: compare polling, SSE, and bidirectional messaging.
- Intermediate: close a subscription when the view unmounts.
- Advanced: disconnect between persistence and notification and recover the current state.
- Challenge: coordinate a snapshot and replay cursor without dropping a concurrent event.

Mini Project

Build a notification-center extension with authenticated streams, stable ids, pagination, duplicate handling, reconnect recovery, and explicit transport limitations.

Project ladder and implementation status

Interview Questions

Beginner: Can SSE send arbitrary client-to-server messages?

SSE is server-to-client; use separate requests for writes.

Intermediate: Is Socket.IO a plain WebSocket protocol?

No. It adds its own protocol and transport behavior.

Advanced: What does presence prove?

Only a bounded observation under the chosen heartbeat and expiry rules.

Scenario: A client reconnects after five missed updates.

Replay from a known position or reconcile from authoritative state.

Debugging: A dashboard repeats an event.

Deduplicate stable event ids and inspect retries and replay boundaries.

Output-based: What does the worked example produce, and under which runtime?

In a browser with a matching authorized SSE endpoint, task-updated messages log the supplied id and version. This is a client fragment; the current workspace does not yet expose that endpoint.

Coding: What would you implement and verify next?

Build a notification-center extension with authenticated streams, stable ids, pagination, duplicate handling, reconnect recovery, and explicit transport limitations.

Architecture and production: Which internal boundary needs evidence?

A transport connection and a durable event log are separate things. Network partitions, process restart, and multiple publishers can change observed ordering. A snapshot followed by subscription can lose intervening writes unless a replay position or reconciliation protocol closes the gap.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Redis caching, messaging, and distributed coordination, System design, consistency, and failure tradeoffs

Next: Redis caching, messaging, and distributed coordination

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Behavioral testing across units, databases, and browsers

Priority: P0 — Essential / Master

Difficulty: Beginner

Importance: 5/5

Prerequisites: JavaScript values, scope, functions, and collections, REST contracts, GraphQL, and API evolution

Estimated learning time: 180 minutes for explanation and initial practice; independent mastery takes repeated application.

Knowledge base · Coverage · Learning paths

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

A unit check exercises a small behavioral contract with controlled inputs and an independent expected outcome.

Concept reference and priorities

Unit testing

P0 · Essential / Master · worked-example

A unit check exercises a small behavioral contract with controlled inputs and an independent expected outcome.

Integration testing

P0 · Essential / Master · reference

Integration checks exercise boundaries between real components such as HTTP parsing and persistence.

API testing

P0 · Essential / Master · reference

API checks verify identity, input, status, representation, ownership, and failure behavior.

Component testing

P1 · ☐ Highly Important · reference

Component checks exercise visible interaction and state without relying unnecessarily on implementation structure.

End-to-end testing

P1 · ☐ Highly Important · reference

Browser workflows exercise important user paths through realistic routing, focus, network, and persistence.

Jest

P2 · Useful · reference

Jest supplies a test runner and mocking ecosystem; configuration and environments affect module behavior.

Vitest

P1 · ☐ Highly Important · reference

Vitest integrates with Vite-oriented projects and offers runner and mocking capabilities.

React Testing Library

P1 · [□](#) [Highly Important](#) · [reference](#)

Testing Library encourages user-oriented queries and interaction rather than component internals.

Supertest

P1 · [□](#) [Highly Important](#) · [reference](#)

Supertest drives HTTP applications through request assertions; its boundary scope must still be stated.

Playwright

P1 · [□](#) [Highly Important](#) · [reference](#)

Playwright automates real browsers with locators, assertions, traces, and workflow tools.

Cypress

P2 · [Useful](#) · [reference](#)

Cypress offers browser-oriented end-to-end and component testing with its own execution model.

Why Does It Exist?

A passing test is useful only when its expected behavior could detect a meaningful failure. Syntax checks and mocks cannot establish database uniqueness or browser focus behavior.

How Does It Work?

Choose the smallest realistic boundary for the risk. Test a reducer without a browser, a unique index against a real database, and modal focus in a real browser. Use accessible roles and names for interaction. Control nondeterministic inputs deliberately; wait for a visible condition rather than sleeping for an arbitrary duration. Make the failure message describe the broken contract.

Examples

1. Trace the contract

```
import test from 'node:test';
import assert from 'node:assert/strict';
test('total includes all positive integer cents', () => {
  const amounts = [1234, 100];
  const total = amounts.reduce((sum, n) => sum + n, 0);
  assert.equal(total, 1334);
});
```

Expected behavior and runtime: Run with `node --test` on a saved module; the assertion passes for 1334. In application tests, call the actual domain function rather than rewriting its implementation in the test.

2. Extend and stress the contract

Intermediate: add a regression check for a reproduced bug. Advanced: run competing writes against a real database. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Mocks replace part of the system and therefore reduce what the test establishes. A fake store cannot prove MongoDB atomic predicates or index behavior. Coverage reports measure execution, not the quality of assertions or missing requirements.

Real-World Usage

Compare toolkit tests, UI reducer tests, MongoDB API tests, and Edge browser smoke tests. State which failure each catches and which dependency remains outside its scope.

Common Mistakes

- Testing that a mocked function returns its own configured result.
- Waiting a fixed timeout for every browser transition.
- Reporting a syntax or unit check as proof that deployment works.

Best Practices

Compare toolkit tests, UI reducer tests, MongoDB API tests, and Edge browser smoke tests. State which failure each catches and which dependency remains outside its scope. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: state expected outputs for three boundary inputs.
- Intermediate: add a regression check for a reproduced bug.
- Advanced: run competing writes against a real database.
- Challenge: verify keyboard focus and data recovery during a network failure in a browser.

Mini Project

Create a test matrix for one workspace feature: pure validation, HTTP contract, persisted invariant, and browser recovery. Remove redundant checks that do not add evidence.

Project ladder and implementation status

Interview Questions

Beginner: What makes an assertion useful?

It compares observed behavior with an independently justified expected result.

Intermediate: When should a real database be used?

When correctness depends on database behavior such as uniqueness, atomicity, or query semantics.

Advanced: What does full coverage not prove?

Correct assertions, complete requirements, realistic data, or operational behavior.

Scenario: Tests pass but users cannot focus a modal control.

Add browser keyboard and focus checks; a simulated DOM may miss the actual interaction.

Debugging: A test intermittently fails.

Inspect shared state, timing assumptions, cleanup, ordering, and uncontrolled external dependencies.

Output-based: What does the worked example produce, and under which runtime?

Run with `node --test` on a saved module; the assertion passes for 1334. In application tests, call the actual domain function rather than rewriting its implementation in the test.

Coding: What would you implement and verify next?

Create a test matrix for one workspace feature: pure validation, HTTP contract, persisted invariant, and browser recovery. Remove redundant checks that do not add evidence.

Architecture and production: Which internal boundary needs evidence?

Mocks replace part of the system and therefore reduce what the test establishes. A fake store cannot prove MongoDB atomic predicates or index behavior. Coverage reports measure execution, not the quality of assertions or missing requirements.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

React identity, state, effects, and resilient interfaces, Express services, validation, and protected REST routes, Containers, CI/CD, and operating a web service

Next: React identity, state, effects, and resilient interfaces

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.

Containers, CI/CD, and operating a web service

Priority: P1 — ☐ Highly Important

Difficulty: Intermediate

Importance: 4/5

Prerequisites: Node.js runtime, resources, and asynchronous services, Git history, collaboration, and code review, Authentication, authorization, and web security boundaries

Estimated learning time: 120 minutes for explanation and initial practice; independent mastery takes repeated application.

[Knowledge base](#) · [Coverage](#) · [Learning paths](#)

Learning Objectives

Explain the mental model, trace the worked example, reproduce a failure, and demonstrate the acceptance checks in the mini-project. Individual concepts below have their own priorities and depth labels.

What Is It?

Linux provides process, filesystem, permission, networking, and service primitives commonly used in deployments.

Concept reference and priorities

Linux

P1 · ☐ Highly Important · [reference](#)

Linux provides process, filesystem, permission, networking, and service primitives commonly used in deployments.

Shell

P1 · ☐ Highly Important · [reference](#)

Shell commands operate on files and processes; quoting and argument boundaries prevent accidental interpretation.

Environment variables

P0 · **Essential / Master** · [reference](#)

Validate configuration at startup and keep secrets separate from source and images.

Docker

P1 · ☐ Highly Important · [reference](#)

Docker packages applications with runtime dependencies using images and managed containers.

Docker Compose

P1 · ☐ Highly Important · [worked-example](#)

Compose defines a multi-service application for a chosen environment; local convenience is not a complete production plan.

Images

P1 · ☐ Highly Important · [reference](#)

Images contain filesystem layers and runtime metadata; build reproducibly and avoid baking secrets into layers.

Containers

P1 · ☐ Highly Important · [reference](#)

Containers are isolated processes sharing host infrastructure, not complete independent virtual machines.

Volumes

P1 · ☐ Highly Important · reference

Volumes persist data beyond a container lifecycle and need backup and restore procedures.

Networks

P1 · ☐ Highly Important · reference

Container networks define communication paths; avoid exposing internal databases unnecessarily.

CI/CD

P1 · ☐ Highly Important · reference

CI validates proposed changes; delivery prepares releases; deployment changes a running environment under a release policy.

GitHub Actions

P1 · ☐ Highly Important · reference

Actions workflows coordinate jobs with explicit triggers, inputs, and permissions.

Nginx

P2 · Useful · reference

Nginx can serve static resources and reverse-proxy HTTP; forwarded-header and timeout configuration need care.

Reverse proxy

P1 · ☐ Highly Important · reference

A proxy sits between clients and services and can handle routing, TLS, limits, or static assets.

SSL/TLS

P1 · ☐ Highly Important · reference

TLS encrypts transport and authenticates endpoints; SSL is historical terminology for obsolete predecessor protocols.

Deployment

P1 · ☐ Highly Important · reference

Deployment includes configuration, migrations, health, rollback, and dependency availability.

Monitoring

P1 · ☐ Highly Important · reference

Monitoring observes signals and actionable conditions; a reachable process is not necessarily ready to serve.

Logging

P1 · ☐ Highly Important · reference

Structured logs support diagnosis while preserving secrecy and limiting volume.

Why Does It Exist?

A successful build only proves that artifacts were created. Operating the service requires configuration, data durability, health, release, and recovery contracts.

How Does It Work?

Create a repeatable image from a lockfile, run as an appropriate non-root user, validate environment, and keep database state in intentional storage. Bind local databases to loopback for learning. CI should exercise meaningful source and database checks before a release is proposed. Separate liveness from readiness so a process that cannot reach its required store does not receive normal traffic.

Examples

1. Trace the contract

```
# Existing workspace, with a running Docker engine
docker compose up -d mongo
docker compose ps
docker compose logs --tail=30 mongo
# Run npm ci, npm test, and npm run build separately as documented.
```

Expected behavior and runtime: These commands start the workspace's local MongoDB service and inspect it. They require Docker Engine/Desktop running. This environment has not established a complete production Compose rollout.

2. Extend and stress the contract

Intermediate: inspect logs and health after a controlled startup failure. Advanced: test a slow request during service replacement. Record the expected result before implementation; use the failure analysis below to distinguish the broken boundary.

Under the Hood

Image layers are reusable filesystem snapshots; containers add runtime process state. Removing a container is distinct from intentionally destroying a volume. Signal handling and HTTP drain behavior determine what happens to in-flight requests during release.

Real-World Usage

Prepare a deployment packet for the workspace with health checks, secret names, persistent-data ownership, migration order, rollback, and a tested restore procedure.

Common Mistakes

- Committing secrets or passing them as permanent image build arguments.
- Treating docker compose up as evidence that backups and rollback work.
- Trusting every forwarded client IP without a proxy configuration.

Best Practices

Prepare a deployment packet for the workspace with health checks, secret names, persistent-data ownership, migration order, rollback, and a tested restore procedure. State the invariant, validate at the boundary, use the documented runtime, and keep a reproducible failure case. This guide is educational evidence; it does not certify a deployment.

Practice

- Beginner: distinguish image, container, and volume.
- Intermediate: inspect logs and health after a controlled startup failure.
- Advanced: test a slow request during service replacement.
- Challenge: restore a disposable backup and compare data rather than assuming the backup is usable.

Mini Project

Add a container and CI packet for the existing source, then verify the image and recovery in an environment with a running engine. Record unexecuted steps honestly.

Project ladder and implementation status

Interview Questions

Beginner: Is a container the same as a VM?

No. Container isolation normally shares host kernel infrastructure.

Intermediate: What does a volume protect?

It separates persistent data lifetime from the container; backup and restore still need design.

Advanced: Why distinguish readiness and liveness?

Readiness controls traffic eligibility; liveness asks whether the process should be restarted.

Scenario: A release runs with missing secrets.

Fail startup validation with a safe diagnostic rather than entering a partially working state.

Debugging: The app works locally but cannot reach MongoDB in Compose.

Inspect service DNS, network, connection URI, port assumptions, and database startup state.

Output-based: What does the worked example produce, and under which runtime?

These commands start the workspace's local MongoDB service and inspect it. They require Docker Engine/Desktop running. This environment has not established a complete production Compose rollout.

Coding: What would you implement and verify next?

Add a container and CI packet for the existing source, then verify the image and recovery in an environment with a running engine. Record unexecuted steps honestly.

Architecture and production: Which internal boundary needs evidence?

Image layers are reusable filesystem snapshots; containers add runtime process state. Removing a container is distinct from intentionally destroying a volume. Signal handling and HTTP drain behavior determine what happens to in-flight requests during release.

Difficulty-based interview bank · Scenario practice · Debugging library

Related Concepts

Cloud deployment, scaling, and recovery choices, Maintainable code, architecture, and observability

Next: Cloud deployment, scaling, and recovery choices

Preserved lessons: notes/docker.md, notes/deployment.md, 09-devops/notes.md

External Resources

Official Documentation

Primary documentation — authoritative reference; use the installed runtime/library version.

Recommended Articles

Focused guide — deepen the mental model and compare its examples with this guide.

Recommended YouTube Videos

See the verified media register. A topic without a verified topical video uses written official documentation; unverified recommendations are not padded into this section.

Practice Resources

Local project ladder · Timed exercises

Further Reading

Curated external register — authority, level, best use, last check, and retrieval limitations.

Last Verified

2026-10-08 — documentation reviewed; example runtime and execution scope are stated above.